

Ph.D Juan David Martínez Vargas
Ph. D Raul Andrés Castañeda

Escuela de Ciencias Aplicadas e Ingeniería

Lecture 02

Deep Learning

Agenda

- Modelos neuronales
- Perceptron: Single layer NNs
- Álgebra lineal y cálculo para Deep Learning

Redes Neuronales: Inspiradas por el Cerebro



<https://www.verywellmind.com/how-brain-cells-communicate-with-each-other-2584397>

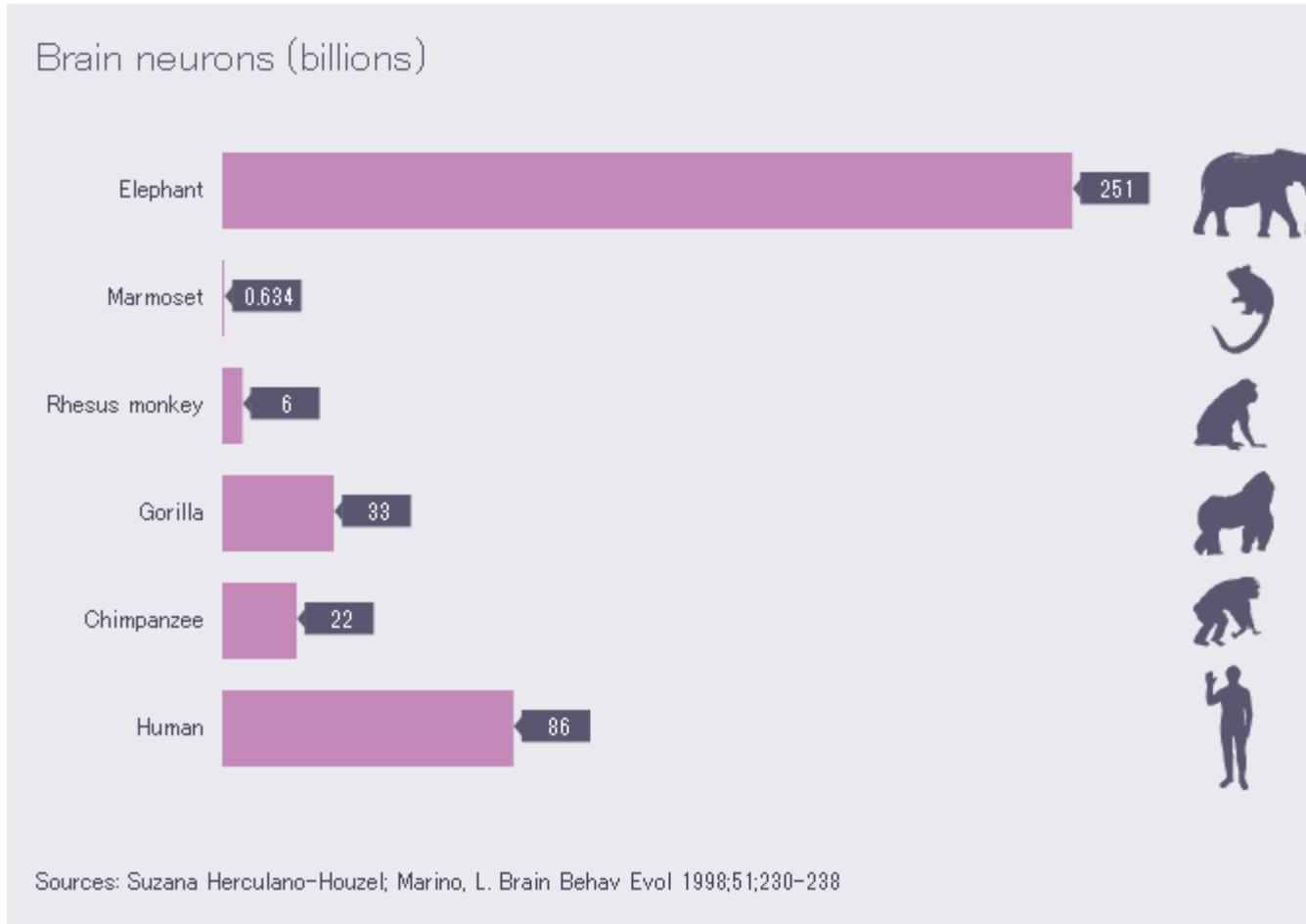
Redes Neuronales: Inspiradas por el Cerebro



<https://medium.com/@adsactly/is-it-a-bird-is-it-a-plane-biomimicry-in-airplanes-9862d331df2e>



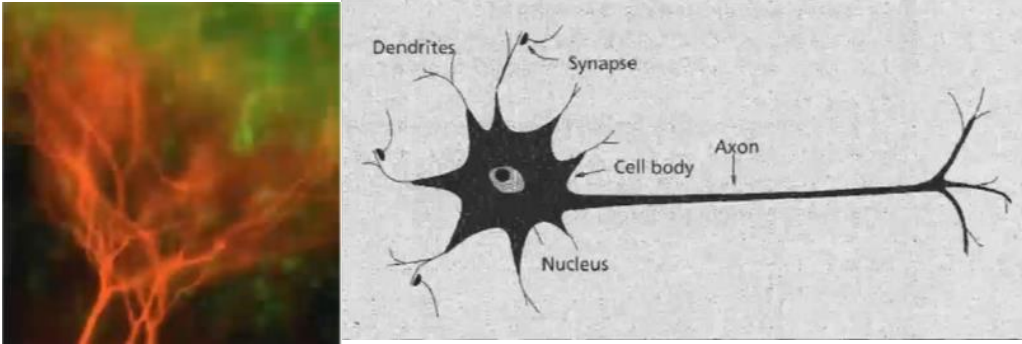
Número de neuronas en el cerebro



- **Dónde están esas neuronas.**
- **Cómo están conectadas.**
- **Densidad neuronal y organización.**

[https://en.wikipedia.org/wiki/List_of_animals_by_number_of_neurons#/media/File:Brain_size_comparison_-_Brain_neurons_\(billions\).png](https://en.wikipedia.org/wiki/List_of_animals_by_number_of_neurons#/media/File:Brain_size_comparison_-_Brain_neurons_(billions).png)

Modelo de Neurona de McCulloch Pitts

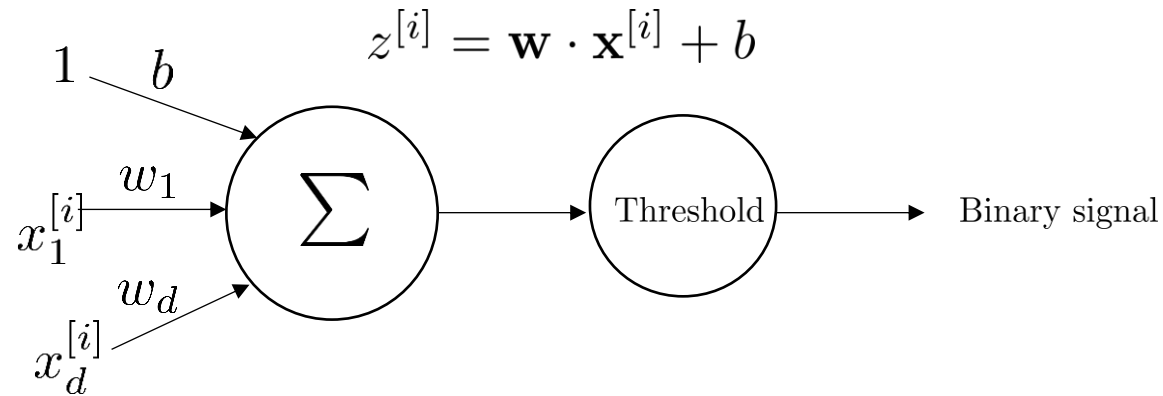
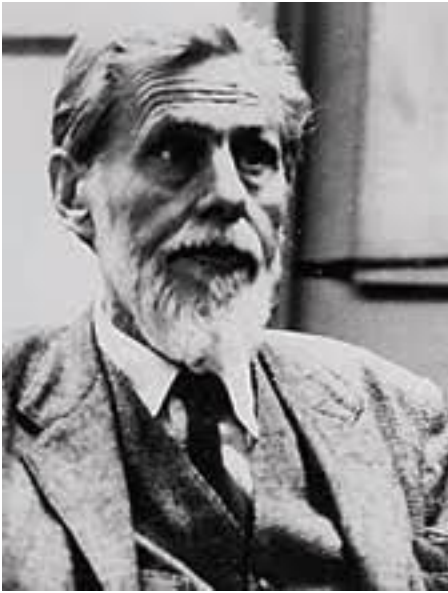


Dendritas: reciben señales de otras neuronas.

Cuerpo celular (soma): integra/suma señales.

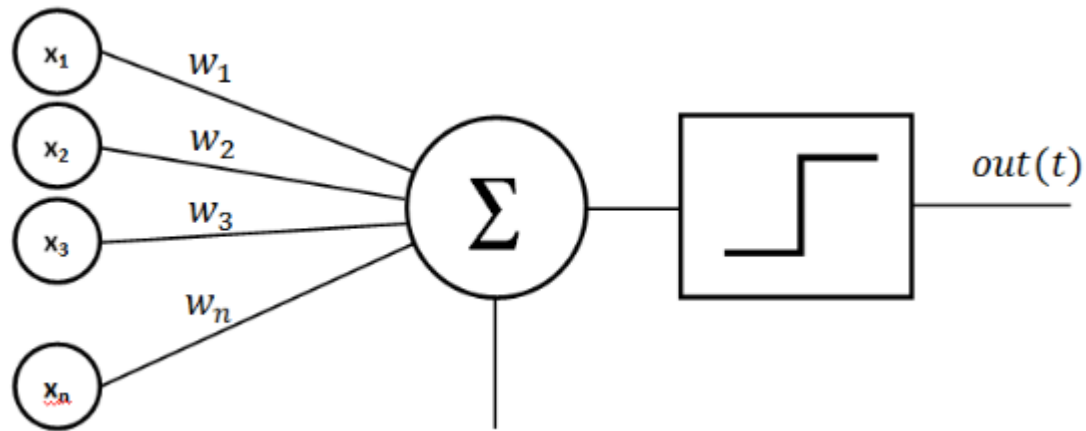
Axón: envía la señal hacia otras neuronas.

Sinapsis: “conexiones” entre neuronas (donde se transmite la señal).



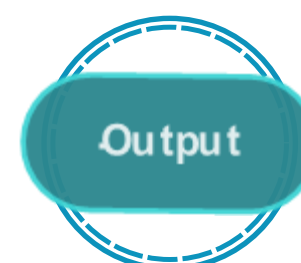
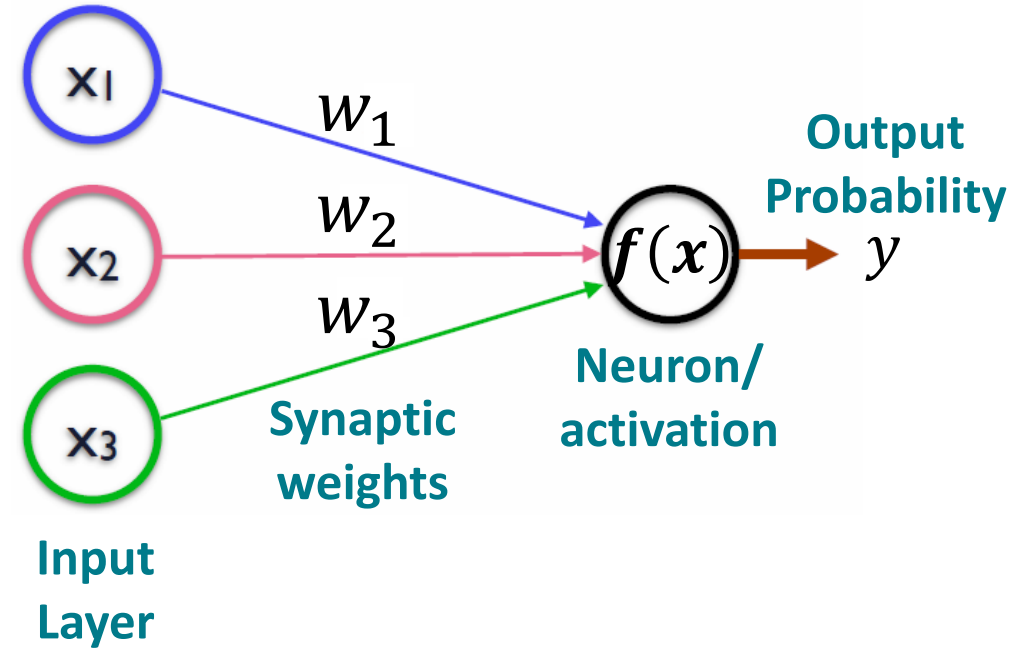
**A LOGICAL CALCULUS OF THE
IDEAS IMMANENT IN NERVOUS ACTIVITY**

Perceptrón

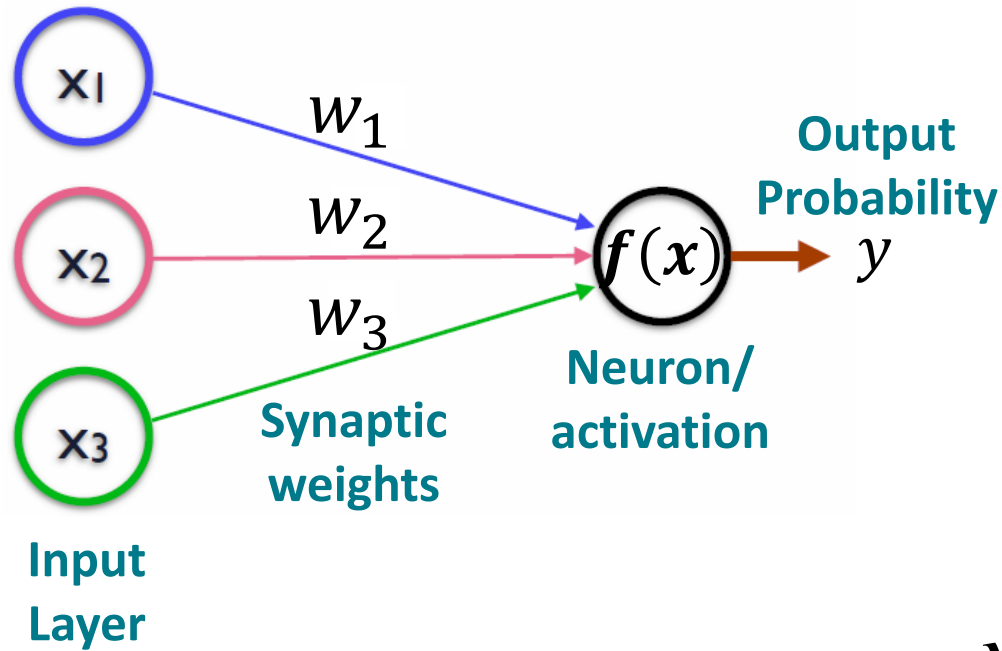


$$\Sigma \rightarrow x_1 w_1 + x_2 w_2 + x_3 w_3 + \dots + x_n w_n$$

$$\sum_{j=1}^n x_j w_j + b_0$$



Perceptrón



$$\Sigma \rightarrow x_1 w_1 + x_2 w_2 + x_3 w_3 + \dots + x_n w_n$$

$$\sum_{j=1}^n x_j w_j + b_0$$

$$f(\mathbf{x}) \rightarrow \begin{cases} a[f(x)] \rightarrow 1; & \text{if } \sum_{j=1}^n x_j w_j \geq \text{Threshold} \\ a[f(x)] \rightarrow -1; & \text{if } \sum_{j=1}^n x_j w_j < \text{Threshold} \end{cases}$$

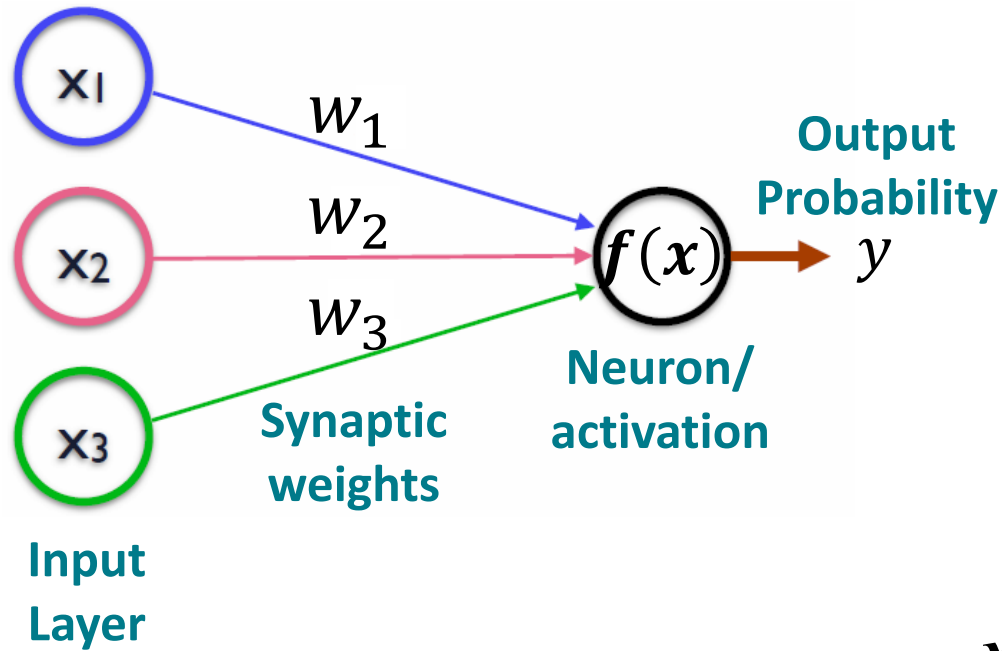
$\mathbf{x} = (x_1, x_2, \dots, x_n)$

Why weights are important?

How do pick up the optimal weights?

The secret for a good model is setup the weight in such a way that the adjusted the weights better understand the input data.

Perceptrón



$$\sum \rightarrow x_1 w_1 + x_2 w_2 + x_3 w_3 + \dots + x_n w_n$$

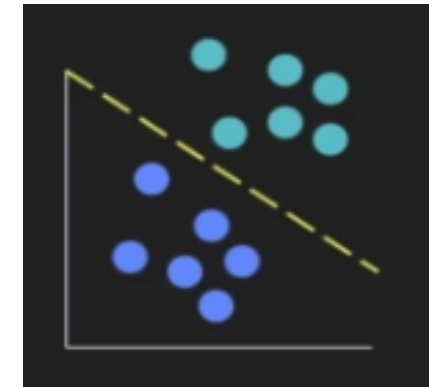
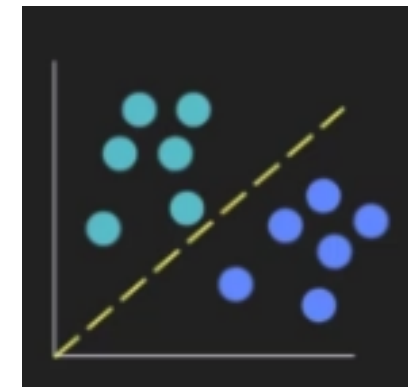
$$\boxed{\sum_{j=1}^n x_j w_j + b_0} \rightarrow f(x) = mx + w_0$$

$$f(\mathbf{x}) \rightarrow \begin{cases} a[f(x)] \rightarrow 1; & \text{if } \sum_{j=1}^n x_j w_j \geq \text{Threshold} \\ a[f(x)] \rightarrow -1; & \text{if } \sum_{j=1}^n x_j w_j < \text{Threshold} \end{cases}$$

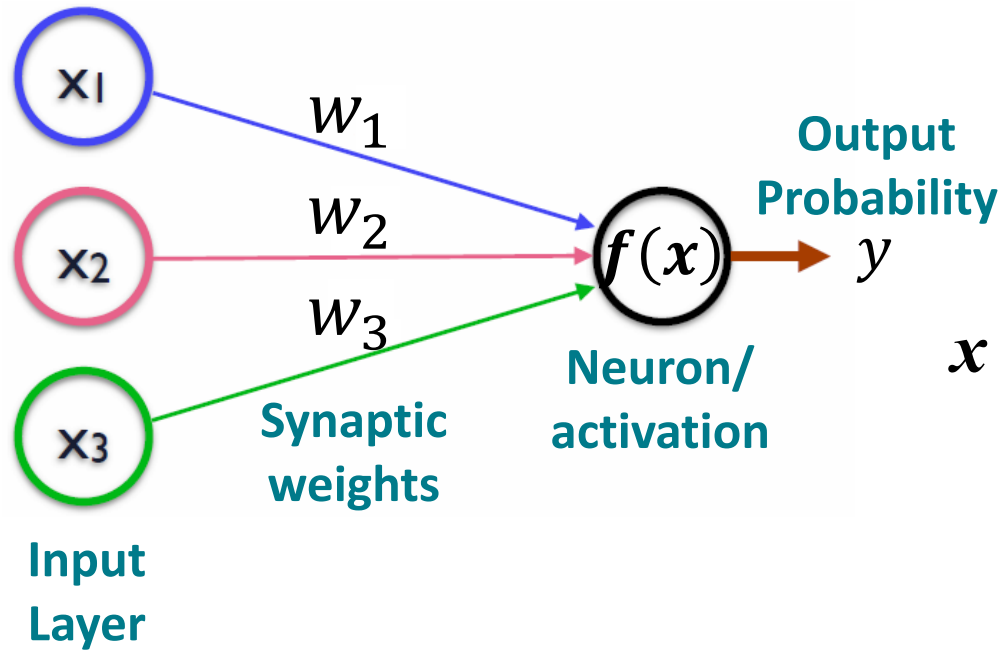
$\mathbf{x} = (x_1, x_2, \dots, x_n)$

Why weights are important?
How do pick up the optimal weights?

The secret for a good model is setup the weight in such a way that the adjusted the weights better understand the input data.

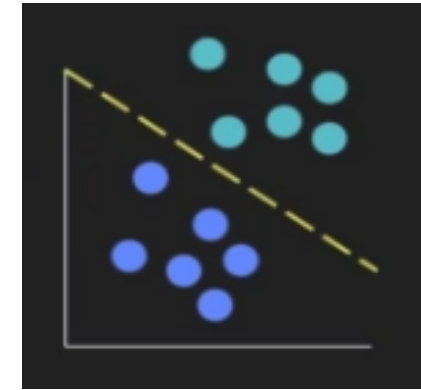
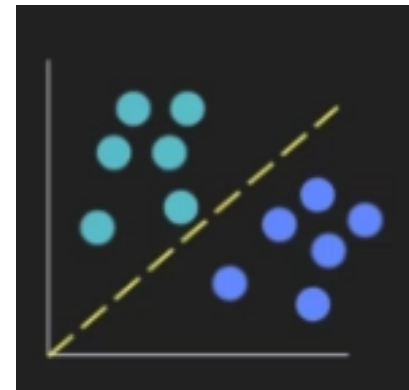


Perceptrón



$$f(\mathbf{x}) \rightarrow \begin{cases} a[f(x)] \rightarrow 1; & \text{if } \sum_{j=1}^n x_j w_j \geq \text{Threshold} \\ a[f(x)] \rightarrow -1; & \text{if } \sum_{j=1}^n x_j w_j < \text{Threshold} \end{cases}$$
$$\mathbf{x} = (x_1, x_2, \dots, x_n)$$

$x_1 \rightarrow$ I have money?
 $x_2 \rightarrow$ I am available?



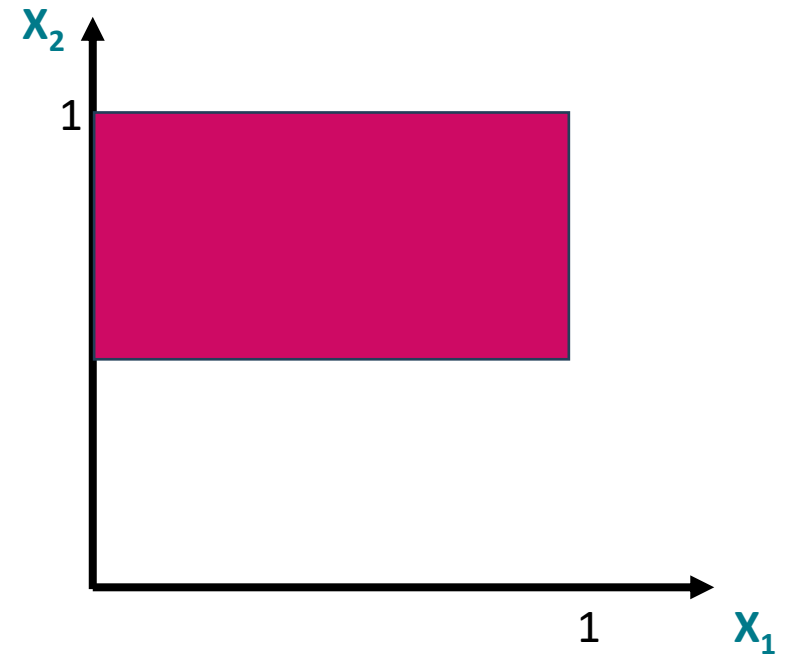
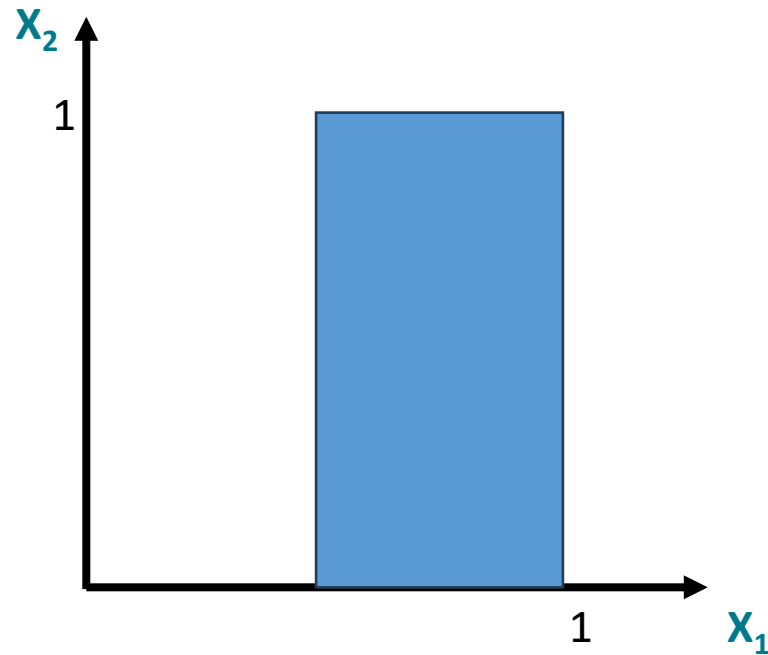
Go to party or not

Neuron / Perceptron

$X_1 \rightarrow$ Do I have money?
 $X_2 \rightarrow$ Do I am available?



Go to party or not

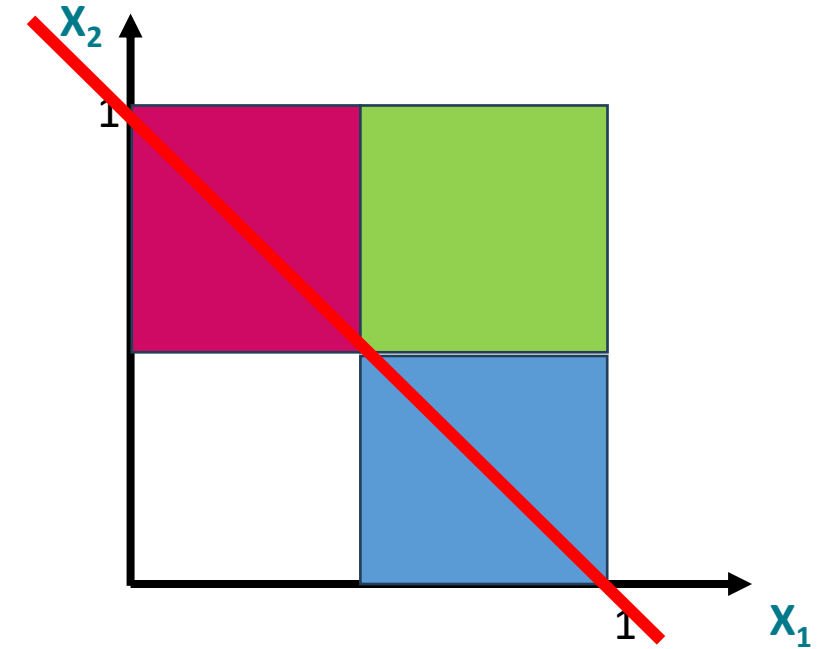
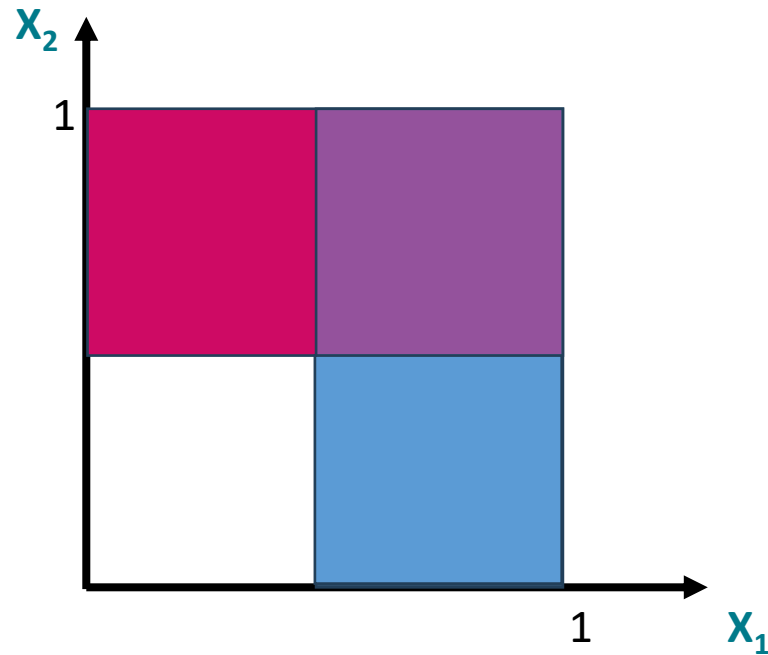


Neuron / Perceptron

$x_1 \rightarrow$ I have money?
 $x_2 \rightarrow$ I am available?



Go to party or not



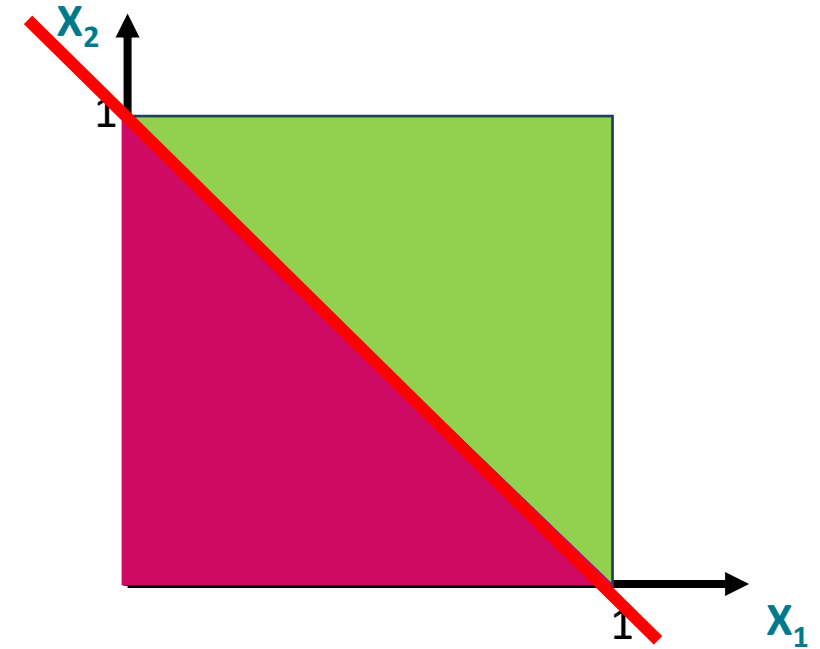
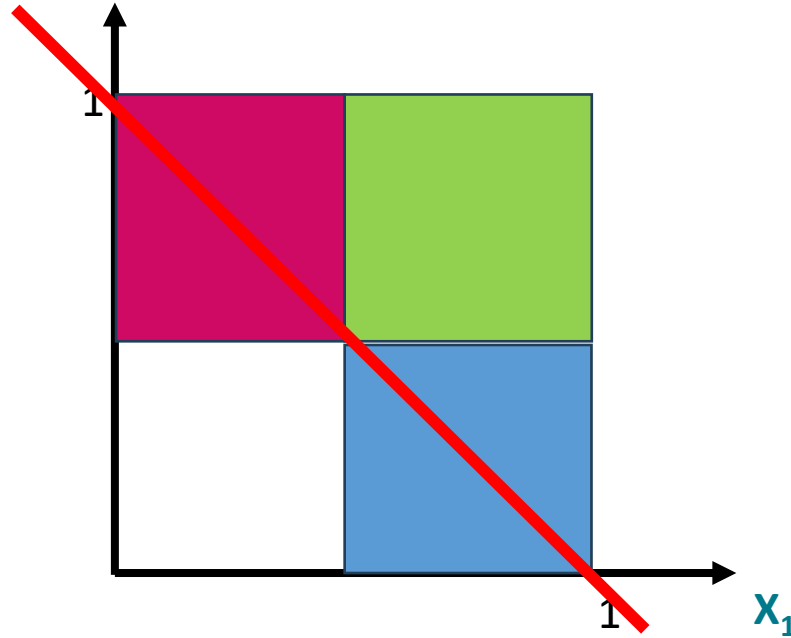
$$\sum_{j=1}^n x_j w_j + b_0$$

Neuron / Perceptron

$x_1 \rightarrow$ I have money?
 $x_2 \rightarrow$ I am available?

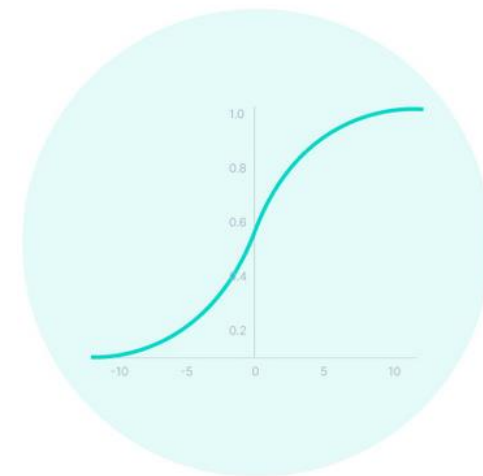


Go to party or not

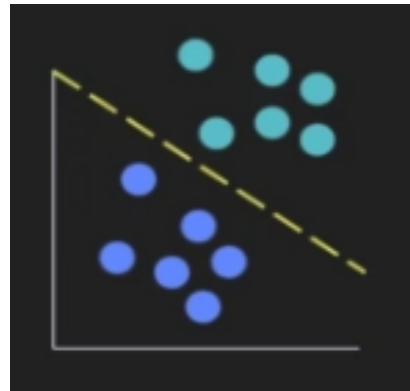
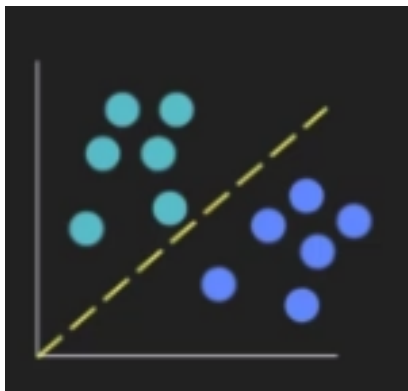
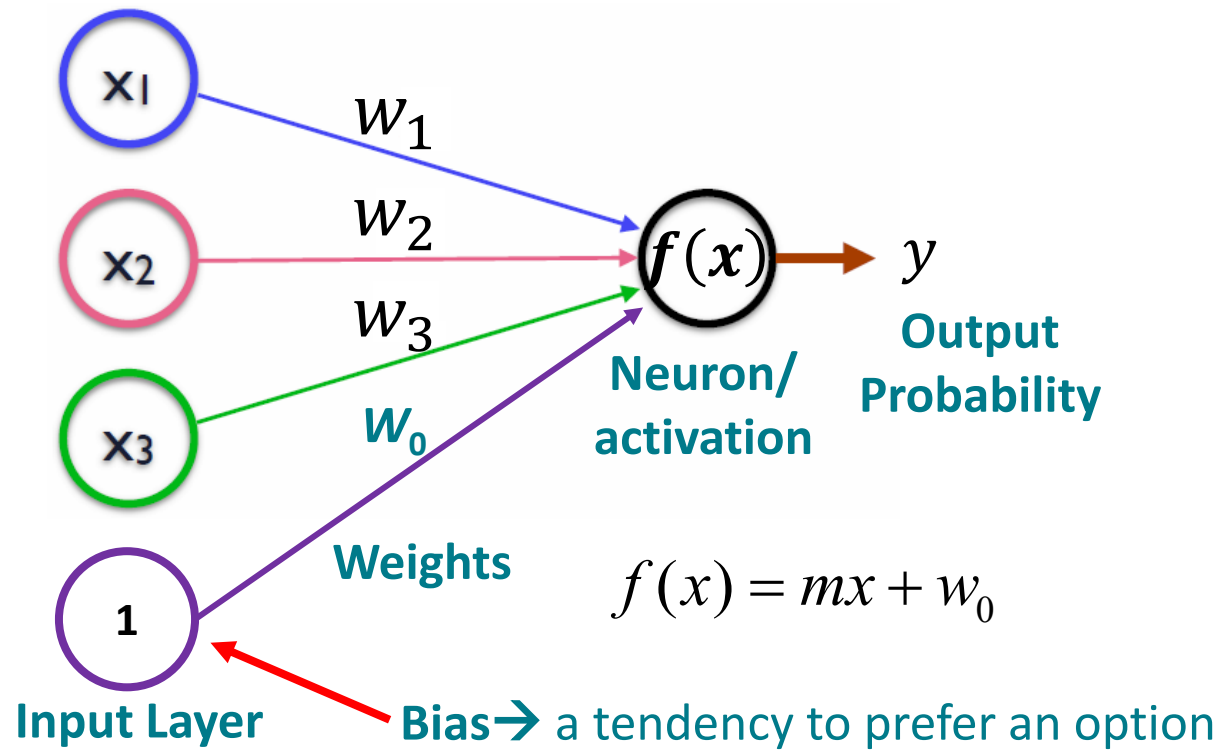


$$\hat{y} = \tau \left[x^T \cdot w + bw_0 \right]$$

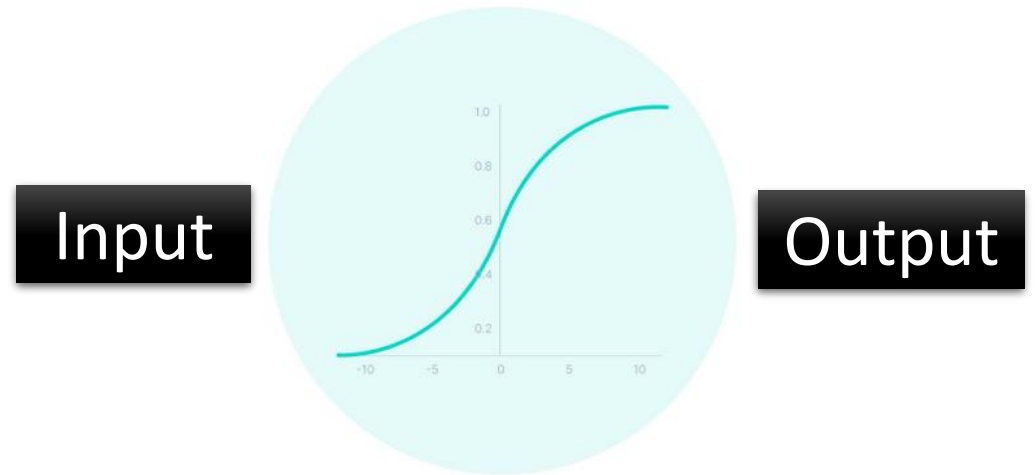
$$\text{Activation} \left[\sum_{i=0}^N x_i w_i \right]$$



Neuron / Perceptron



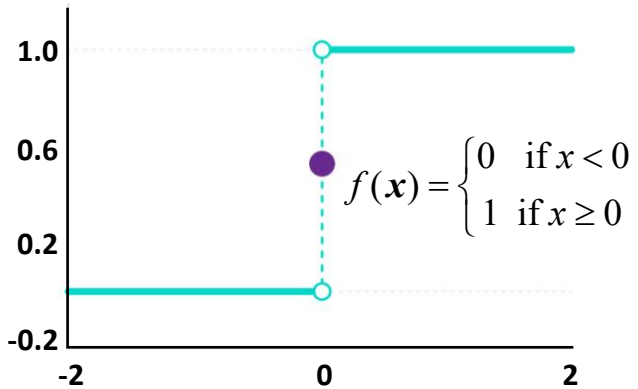
$$\text{Activation} \left[\sum_{i=0}^N x_i w_i \right]$$



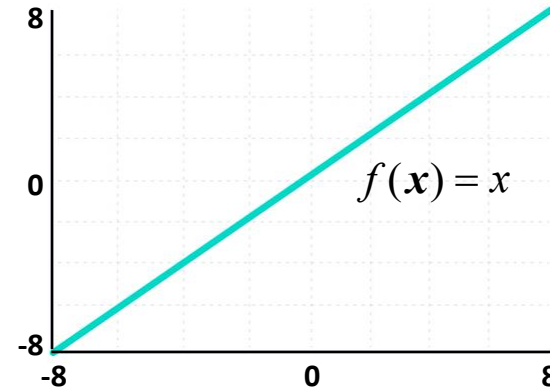
Activation function

Applied to the weighted sum to decide whether to send this signal to the next layer or not

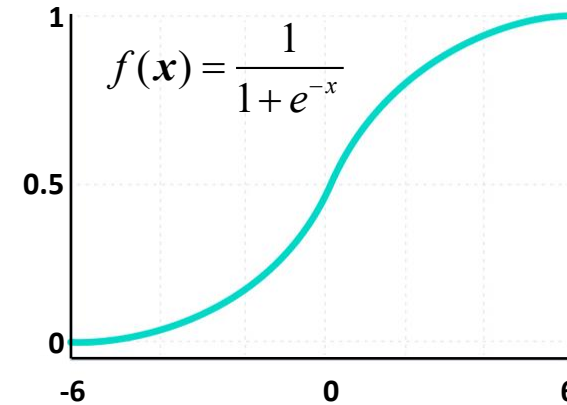
Binary Step Function



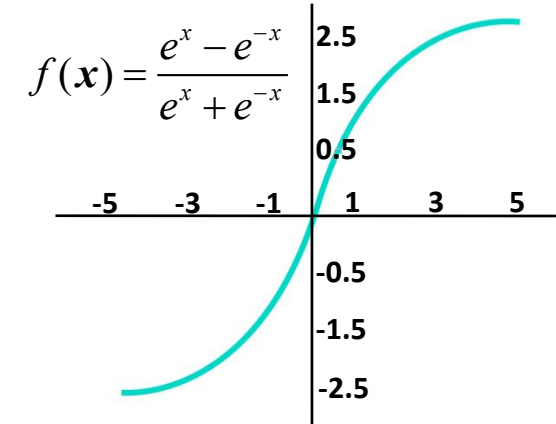
Linear Activation Function



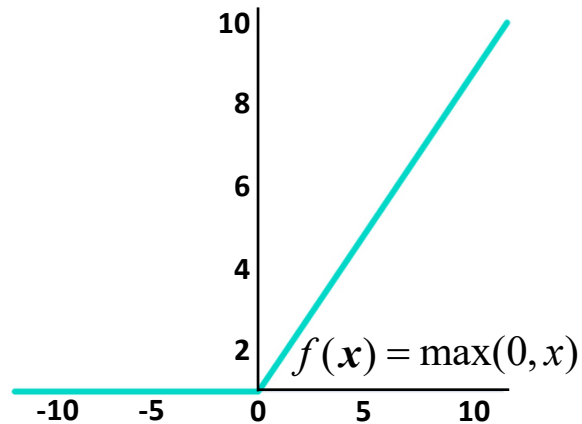
Sigmoid / Logistic



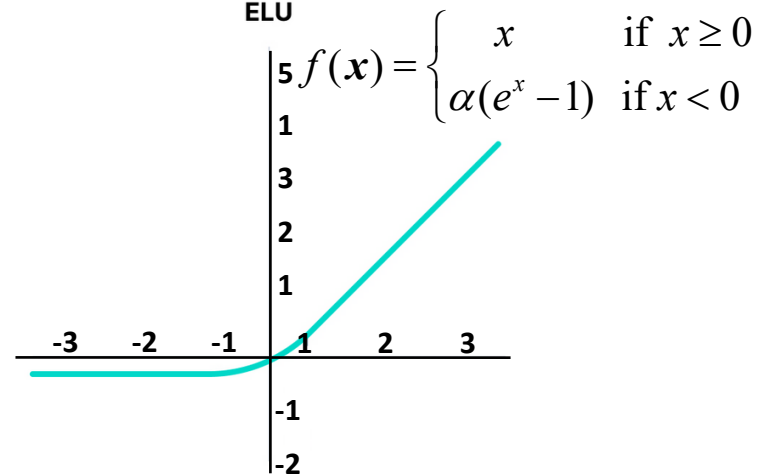
Tanh



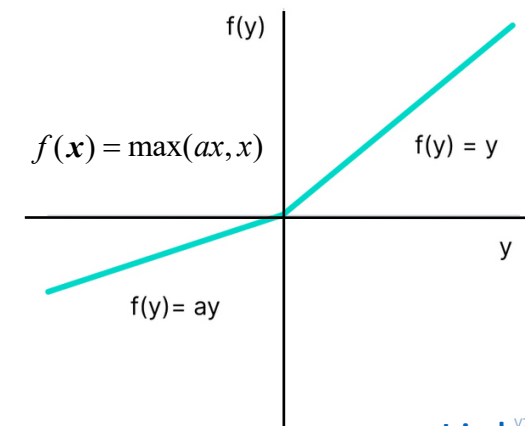
ReLU



ELU



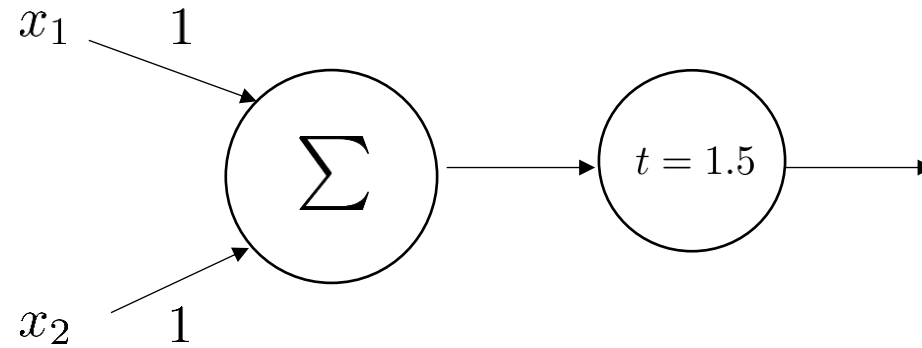
Parametric ReLU



[Link](#)

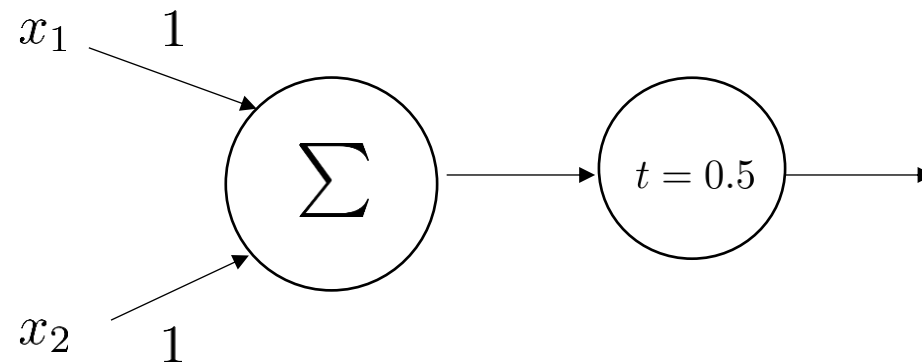
And

x_1	x_2	Out
0	0	0
0	1	0
1	0	0
1	1	1



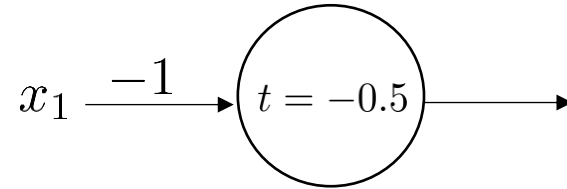
Or

x_1	x_2	Out
0	0	0
0	1	1
1	0	1
1	1	1



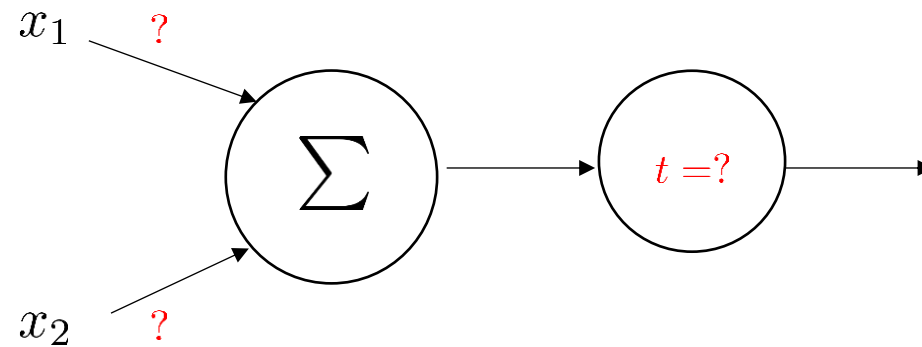
Not

x_1	Out
0	1
1	0



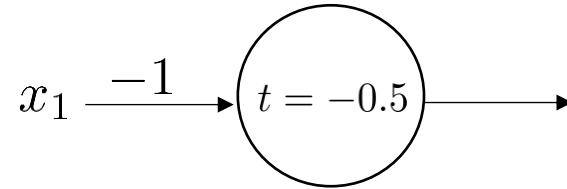
Xor

x_1	x_2	Out
0	0	0
0	1	1
1	0	1
1	1	0



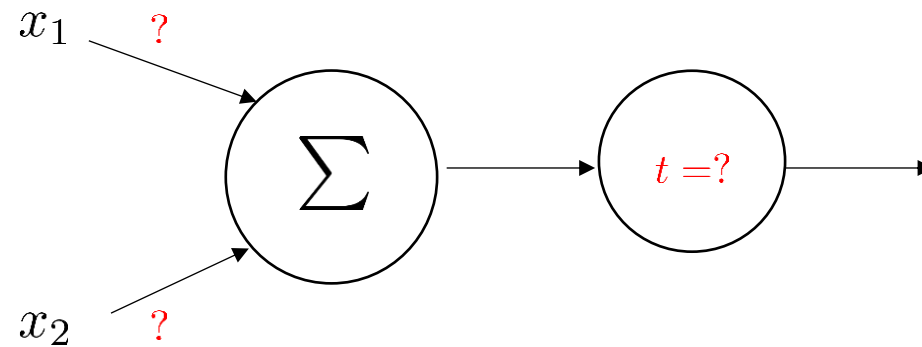
Not

x_1	Out
0	1
1	0

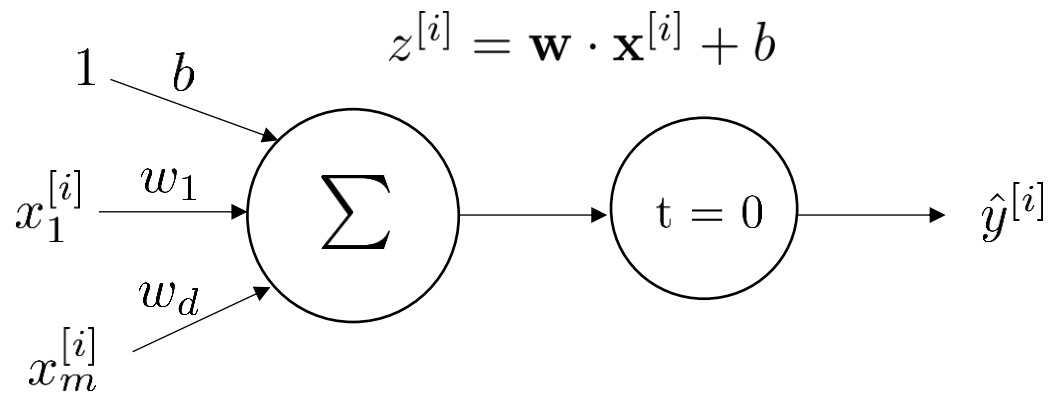


Xor

x_1	x_2	Out
0	0	0
0	1	1
1	0	1
1	1	0

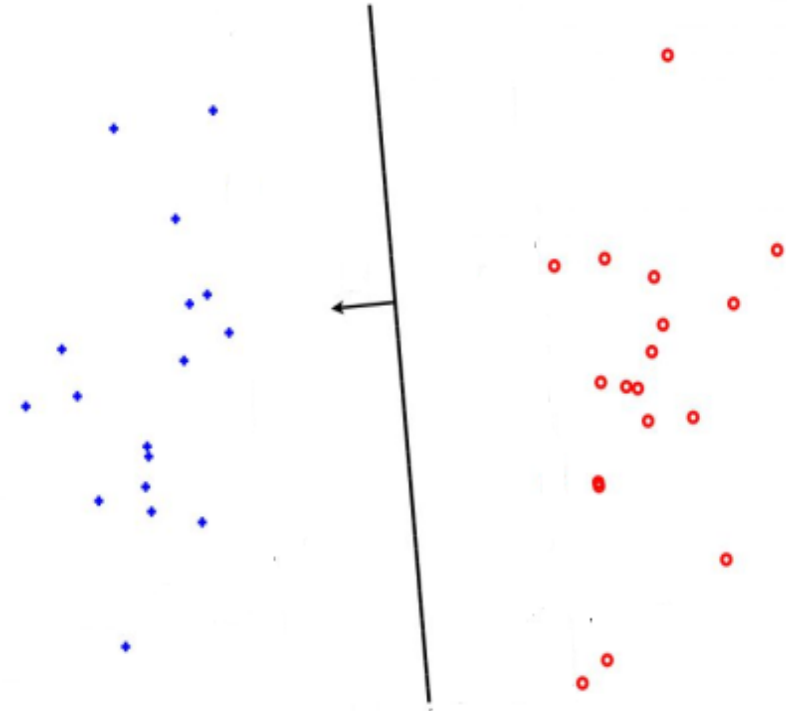


Perceptrón



$$\mathbf{x}^{[i]} = [1, x_1^{[i]}, \dots, x_m^{[i]}] \quad w_0 = b$$

$$z^{(i)} = \mathbf{w} \cdot \mathbf{x}^{[i]}$$



Compuertas Lógicas

AND

Inputs		Output
x1	x2	y
0	0	0
0	1	0
1	0	0
1	1	1

OR

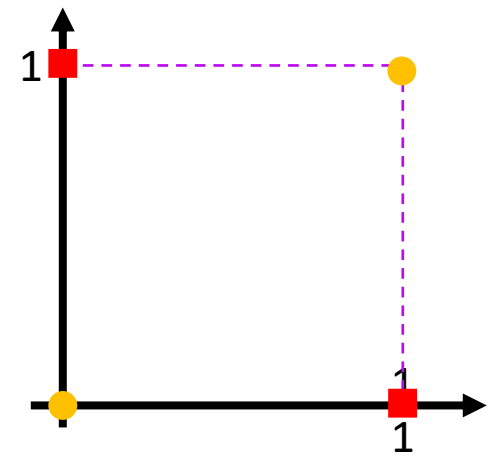
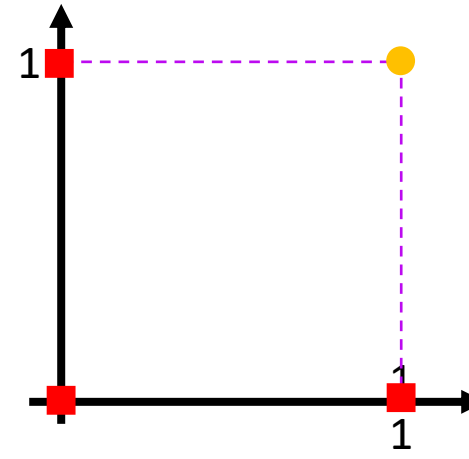
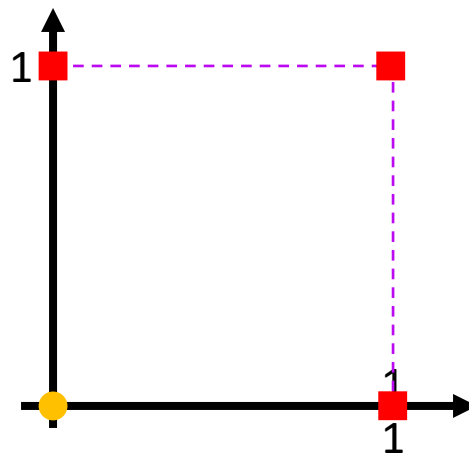
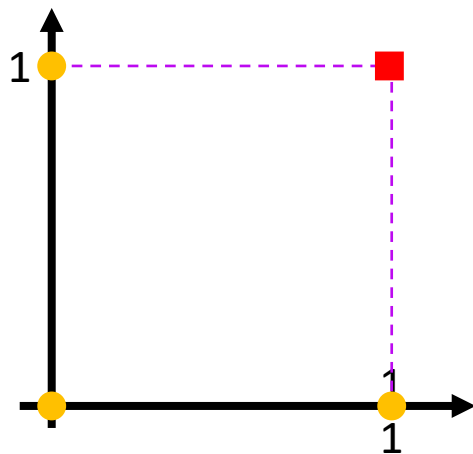
Inputs		Output
x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	1

NAND

Inputs		Output
x1	x2	y
0	0	1
0	1	1
1	0	1
1	1	0

XOR

Inputs		Output
x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0



Compuertas Lógicas

AND

Inputs		Output
x1	x2	y
0	0	0
0	1	0
1	0	0
1	1	1

OR

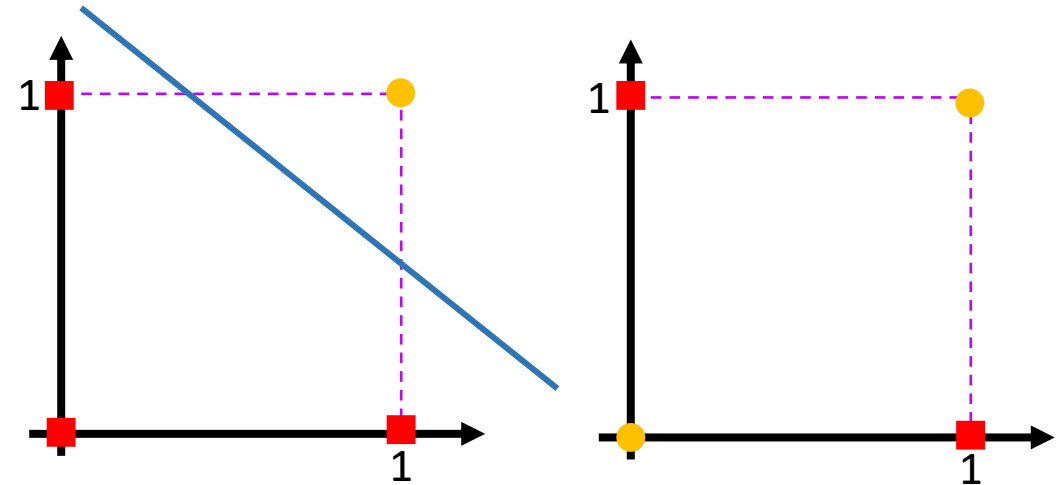
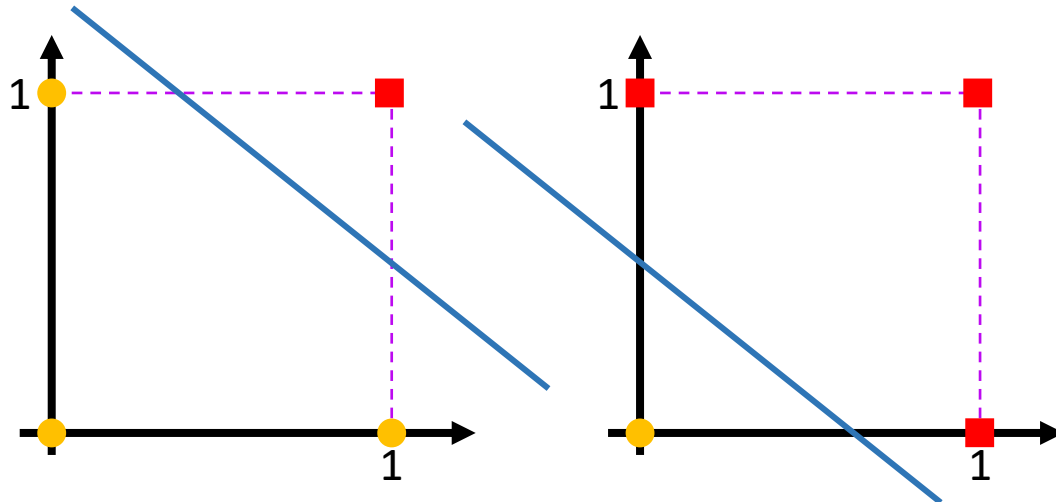
Inputs		Output
x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	1

NAND

Inputs		Output
x1	x2	y
0	0	1
0	1	1
1	0	1
1	1	0

XOR

Inputs		Output
x1	x2	y
0	0	0
0	1	1
1	0	1
1	1	0



$$\mathcal{D} = \{(\mathbf{x}^{[i]}, y^{[i]}) | i = 1, \dots, n\} \quad \mathbf{x}^{[i]} \in \mathbb{R}^{m+1} \quad y^{[i]} \in \{0, 1\}$$

Result: $\mathbf{w}$

$\mathbf{w} = \mathbf{0}_{m+1};$

for $t = 1, \dots, T$ **do**

for $i = 1, \dots, n$ **do**

 instructions;

if $y^{[i]}(\mathbf{w} \cdot \mathbf{x}^{[i]}) \leq 0$ **then**

$\mathbf{w} = \mathbf{w} + y^{[i]}\mathbf{x}^{[i]}$

else

 do nothing

end

end

end

$$\mathcal{D} = \{(\mathbf{x}^{[i]}, y^{[i]}) | i = 1, \dots, n\} \quad \mathbf{x}^{[i]} \in \mathbb{R}^{m+1} \quad y^{[i]} \in \{0, 1\}$$

Result: $\mathbf{w}$

```
 $\mathbf{w} = \mathbf{0}_{m+1};$ 
for  $t = 1, \dots, T$  do
  for  $i = 1, \dots, n$  do
    instructions;
    if  $y^{[i]}(\mathbf{w} \cdot \mathbf{x}^{[i]}) \leq 0$  then
       $\mathbf{w} = \mathbf{w} + y^{[i]}\mathbf{x}^{[i]}$ 
    else
      do nothing
    end
  end
end
```

Este código implementa el **algoritmo Perceptrón**, un clasificador binario lineal que aprende iterativamente ajustando los pesos

Si $y^{[i]} \cdot (\mathbf{w} \cdot \mathbf{x}^{[i]}) \leq 0 \rightarrow$ **mal clasificado** $\rightarrow$ actualiza $\mathbf{w}$

Si $y^{[i]} \cdot (\mathbf{w} \cdot \mathbf{x}^{[i]}) > 0 \rightarrow$ **bien clasificado** $\rightarrow$ no hace nada

Como principio general de aprendizaje, aplicable a todos los modelos comunes de neuronas y arquitecturas de redes neuronales (profundas y no profundas):

Sea

$$\mathcal{D} = ((\mathbf{x}^{[1]}, y^{[1]}), (\mathbf{x}^{[2]}, y^{[2]}), \dots, (\mathbf{x}^{[n]}, y^{[n]})) \in (\mathbb{R}^m \times \{0, 1\})^n$$

Modo *On-line*

Result: $\mathbf{w}, b$

$\mathbf{w} := \mathbf{0} \in \mathbb{R}^m, \mathbf{b} := 0;$

for $t = 1, \dots, T$ **do**

for $i = 1, \dots, n$ **do**

 cálculo de la salida;

 cálculo del error ;

 actualización de parámetros $\mathbf{w}, b$;

end

end

Modo *On-line*

```
Result:  $\mathbf{w}, b$   
 $\mathbf{w} := \mathbf{0} \in \mathbb{R}^m, \mathbf{b} := 0;$   
for  $t = 1, \dots, T$  do  
  for  $i = 1, \dots, n$  do  
    cálculo de la salida;  
    cálculo del error ;  
    actualización de  
      parámetros  $\mathbf{w}, b$ ;  
  end  
end
```

Modo *On-line* II

```
Result:  $\mathbf{w}, b$   
 $\mathbf{w} := \mathbf{0} \in \mathbb{R}^m, \mathbf{b} := 0;$   
for  $j$  iteraciones do  
  Elija un  $(\mathbf{x}^{[i]}, y^{[i]}) \in \mathcal{D}$   
  aleatorio;  
  cálculo de la salida;  
  cálculo del error ;  
  actualización  $\mathbf{w}, b$ ;  
end
```

Modo *Batch*

Modo *On-line*

```
Result:  $\mathbf{w}, b$ 
 $\mathbf{w} := \mathbf{0} \in \mathbb{R}^m, \mathbf{b} := 0;$ 
for  $t = 1, \dots, T$  do
  for  $i = 1, \dots, n$  do
    cálculo de la salida;
    cálculo del error ;
    actualización de
      parámetros  $\mathbf{w}, b;$ 
  end
end
```

```
Result:  $\mathbf{w}, b$ 
 $\mathbf{w} := \mathbf{0} \in \mathbb{R}^m, \mathbf{b} := 0;$ 
for  $t = 1, \dots, T$  do
   $\Delta \mathbf{w} := 0, \Delta b := 0;$ 
  for  $i = 1, \dots, n$  do
    cálculo de la salida;
    cálculo del error ;
    actualización de
      parámetros  $\Delta \mathbf{w}, \Delta b;$ 
  end
  actualización de parámetros
     $\mathbf{w}, b;$ 
   $\mathbf{w} := \mathbf{w} + \Delta \mathbf{w} \quad b := b + \Delta b;$ 
end
```



Modo *minibatch*

Modo más común en Deep Learning. Combina *On-line* y *Batch*.

$$\mathcal{D} = ((\mathbf{x}^{[1]}, y^{[1]}), (\mathbf{x}^{[2]}, y^{[2]}), \dots, (\mathbf{x}^{[n]}, y^{[n]})) \in (\mathbb{R}^m \times \{0, 1\})^n$$

Result: $\mathbf{w}, b$

$\mathbf{w} := \mathbf{0} \in \mathbb{R}^m, \mathbf{b} := 0;$

for $t = 1, \dots, T$ **do**

for $j = 1, \dots, n/k$ **do**

$\Delta \mathbf{w} := 0, \Delta b := 0;$

for $\{(\mathbf{x}^{[i]}, y^{[i]}), \dots, (\mathbf{x}^{[i+k]}, y^{[i+k]})\} \subset D$ **do**

 cálculo de la salida;

 cálculo del error ;

 actualización de $\Delta \mathbf{w}, \Delta b;$

end

 actualización $\mathbf{w}, b;$

$\mathbf{w} := \mathbf{w} + \Delta \mathbf{w};$

$b := b + \Delta b;$

end

end



Implementación

```
3
4 class Perceptron():
5     def __init__(self, num_features):
6         self.num_features = num_features
7         self.weights = np.zeros((num_features, 1), dtype=np.float)
8         self.bias = np.zeros(1, dtype=np.float)
9
10
11
12
13
14
15
16
17
18     def forward(self, x):
19         linear = np.dot(x, self.weights) + self.bias
20         predictions = np.where(linear > 0., 1, 0)
21         return predictions
22
23     def backward(self, x, y):
24         predictions = self.forward(x)
25         errors = y - predictions
26         return errors
27
28     def train(self, x, y, epochs):
29         for e in range(epochs):
30
31             for i in range(y.shape[0]):
32
33                 errors = self.backward(x[i].reshape(1, self.num_features),
34 y[i]).reshape(-1)
35                 self.weights += (errors * x[i]).reshape(self.num_features,
36 1)
37                 self.bias += errors
38
39     def evaluate(self, x, y):
40         predictions = self.forward(x).reshape(-1)
41         accuracy = np.sum(predictions == y) / y.shape[0]
42         return accuracy
```

```
3
4 class Perceptron():
5     def __init__(self, num_features):
6         self.num_features = num_features
7
8
9         self.weights = torch.zeros(num_features, 1,
10                                     dtype=torch.float32, device=device)
11         self.bias = torch.zeros(1, dtype=torch.float32, device=device)
12
13         # placeholder vectors so they don't
14         # need to be recreated each time
15         self.ones = torch.ones(1)
16         self.zeros = torch.zeros(1)
17
18     def forward(self, x):
19         linear = torch.add(torch.mm(x, self.weights), self.bias)
20         predictions = torch.where(linear > 0., self.ones, self.zeros)
21         return predictions
22
23     def backward(self, x, y):
24         predictions = self.forward(x)
25         errors = y - predictions
26         return errors
27
28     def train(self, x, y, epochs):
29         for e in range(epochs):
30
31             for i in range(y.shape[0]):
32
33                 # use view because backward expects a matrix (i.e., 2D tensor)
34                 errors = self.backward(x[i].reshape(1, self.num_features),
35 y[i]).reshape(-1)
36                 self.weights += (errors * x[i]).reshape(self.num_features,
37 1)
38                 self.bias += errors
39
40     def evaluate(self, x, y):
41         predictions = self.forward(x).reshape(-1)
42         accuracy = torch.sum(predictions == y).float() / y.shape[0]
43         return accuracy
```

Un modelo puede tener 100% accuracy y aun así estar aprendiendo la razón equivocada

[...] Where a perceptron had been trained to distinguish between - this was for military purposes - it was looking at a scene of a forest in which there were camouflaged tanks in one picture and no camouflaged tanks in the other. And the perceptron - after a little training - made a 100% correct distinction between these two different sets of photographs. Then they were embarrassed a few hours later to discover that the two rolls of film had been developed differently. And so these pictures were just a little darker than all of these pictures and the perceptron was just measuring the total amount of light in the scene. But it was very clever of the perceptron to find some way of making the distinction.

-- Marvin Minsky, AI researcher & author of the "Perceptrons" book

Source: <https://www.webofstories.com/play/marvin.minsky/122>



<https://qph.fs.quoracdn.net/main-qimg-305eb8136c4a20f348bb7ab465bc2e10>

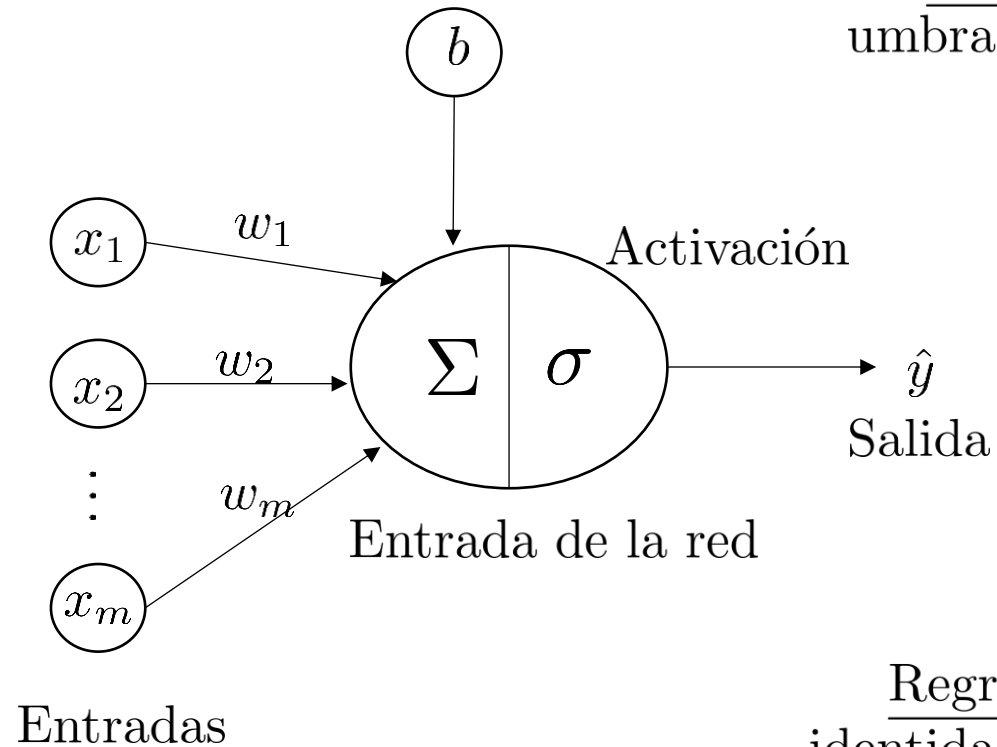


<http://theconversation.com/want-to-beat-climate-change-protect-our-natural-forests-121491>

Un modelo puede tener 100% accuracy y aun así estar aprendiendo la razón equivocada

Regresión lineal

Perceptrón: la función de activación es la función de umbral. La salida es la etiqueta binaria $\hat{y} \in \{0, 1\}$



Regresión lineal: la función de activación es la función identidad $\sigma(x) = x$. La salida es el número $\hat{y} \in \mathbb{R}$

Regresión Lineal

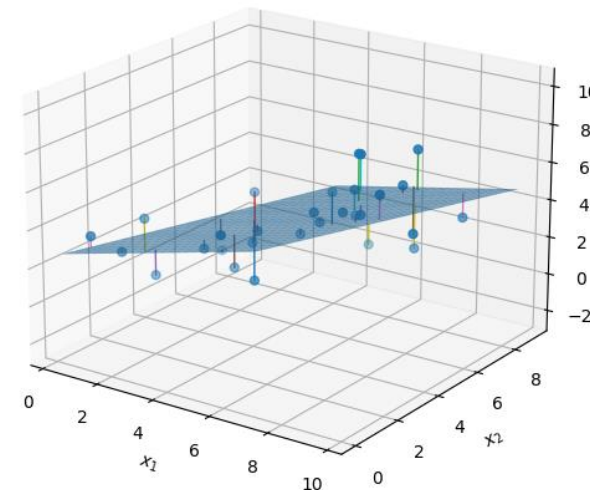
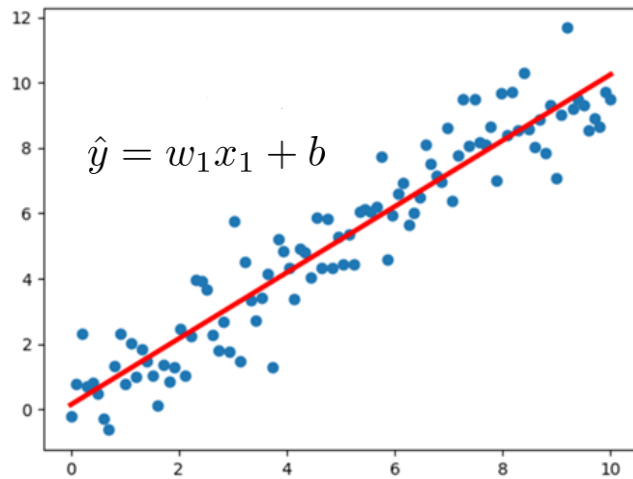
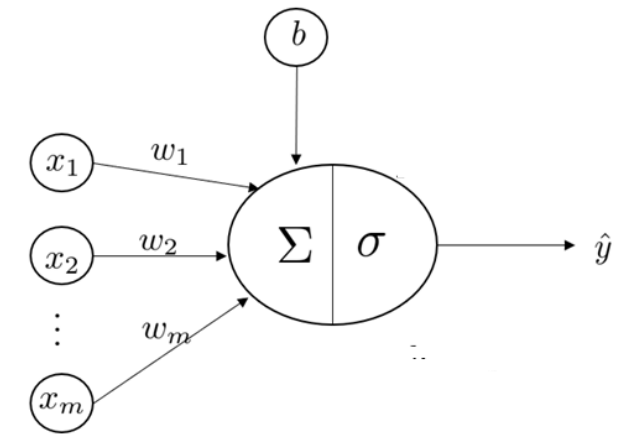
$$\hat{y} = w_1 x_1 + b$$

Describe la relación entre una variable dependiente $\hat{y}$ y una única variable independiente x .

Regresión Lineal Múltiple

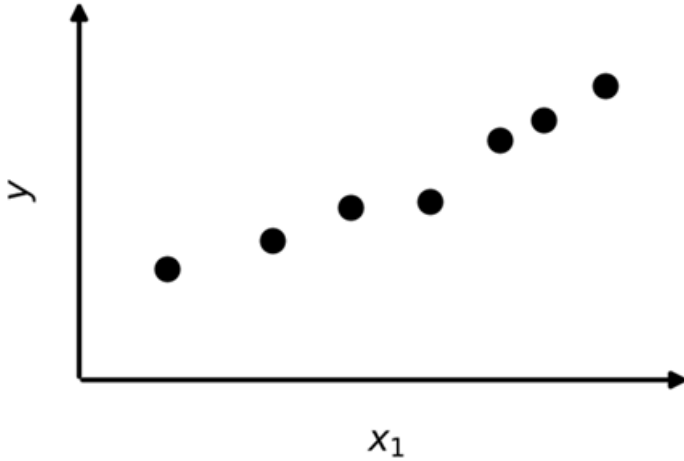
$$\hat{y} = b + w_1 x_1 + w_2 x_2 + \dots + w_m x_m$$

Relaciona una variable dependiente $\hat{y}$ con dos o más variables independientes.

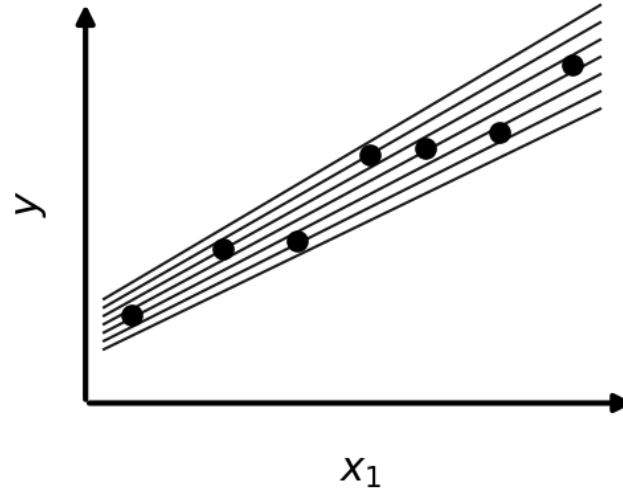


Regresión lineal

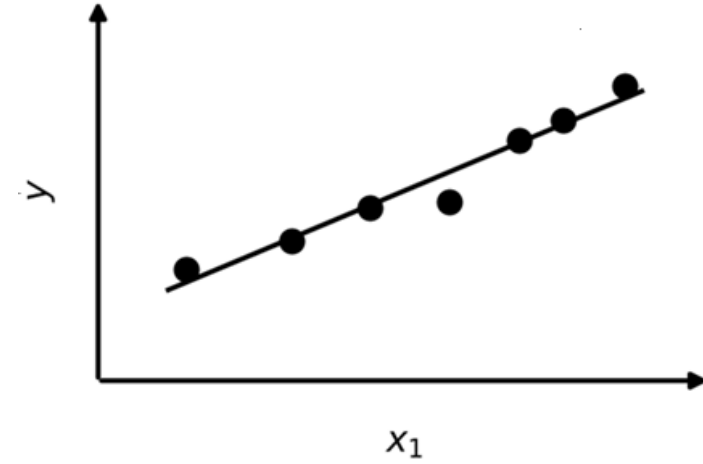
Conjunto de datos



Posibles ajustes



Mejor línea

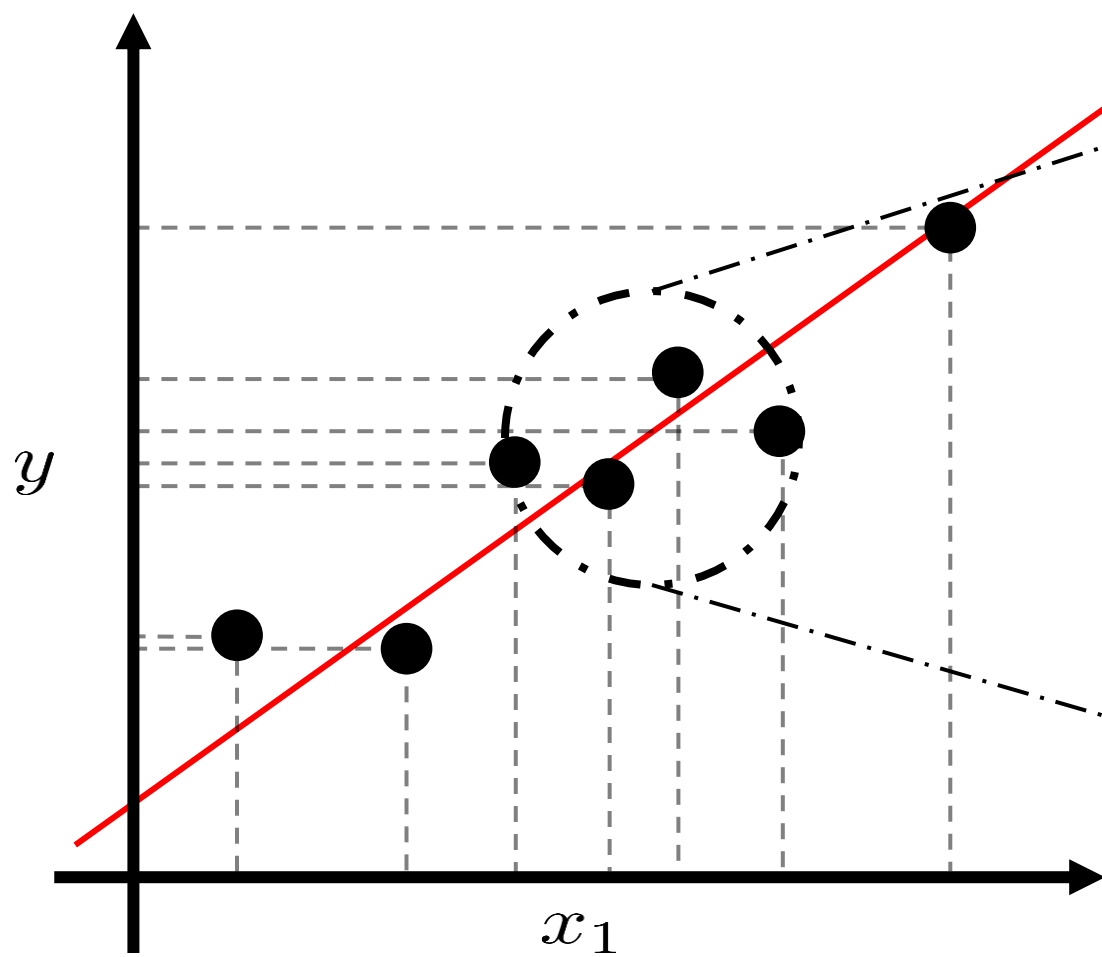


¿Cuál es la línea que mejor se ajusta a los datos?

Minimizar la distancia entre la línea y cada punto

Diferencia entre el valor predicho y el valor real

Regresión lineal

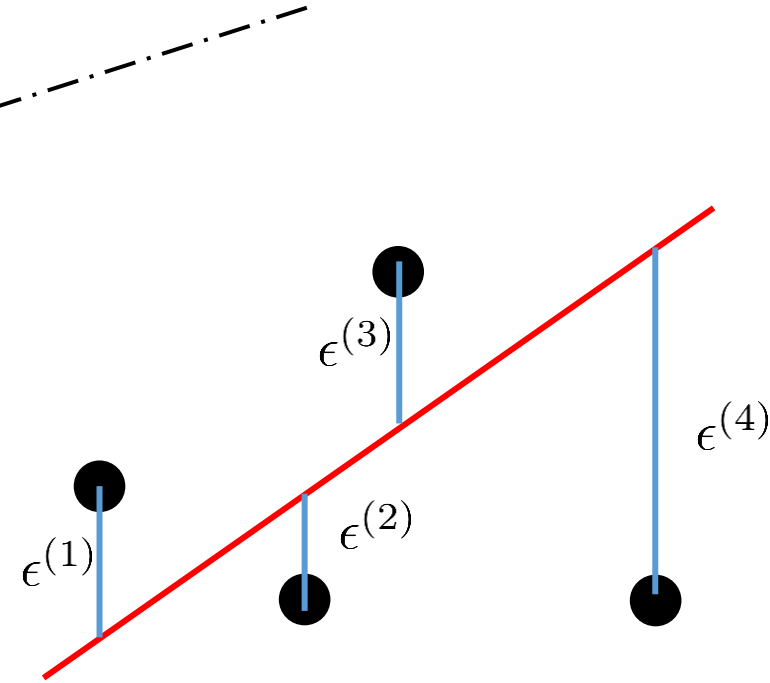


$$\hat{y} = w_1 x_1 + b$$

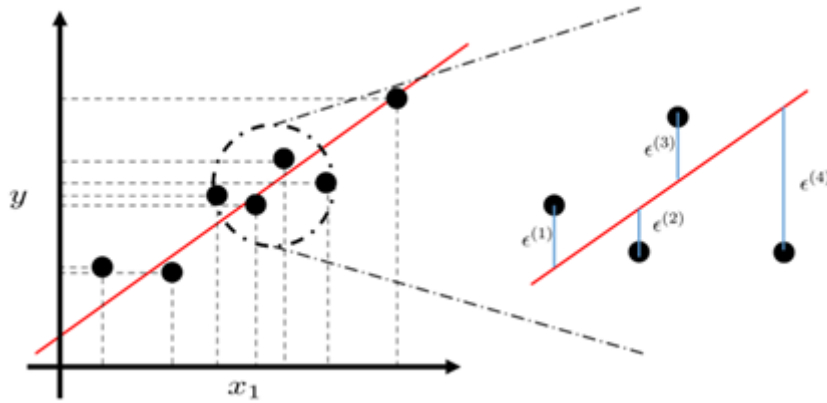


$$\epsilon^{(i)} = y^{(i)} - \hat{y}^{(i)}$$

$$\epsilon^{(i)} = y^{(i)} - (w_1 x_1^{(i)} + b)$$



Regresión lineal



Minimizar la distancia entre la línea y cada punto

Función de costo:

$$L(w_1, b) = \frac{1}{n} \sum_{i=1}^n (y^{(i)} - (w_1 x^{(i)} + b))^2$$

$$L(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n (y^{(i)} - (\mathbf{w}^\top \mathbf{x}^{(i)} + b))^2$$

$L(w_1, b)$ es la función de pérdida (loss): **mide el error promedio cuadrático entre lo real y lo que predice el modelo lineal** $w^T x + b$.

Regresión lineal

ε_i es el error por dato. La pérdida cuadrática es $(\varepsilon_i)^2$.
La función de costo J es la suma (o el promedio) de esas pérdidas en todo el dataset.

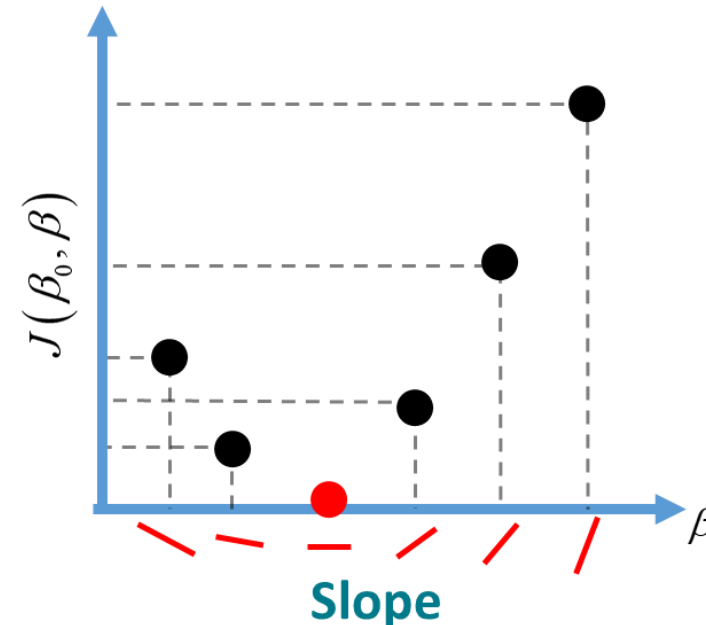
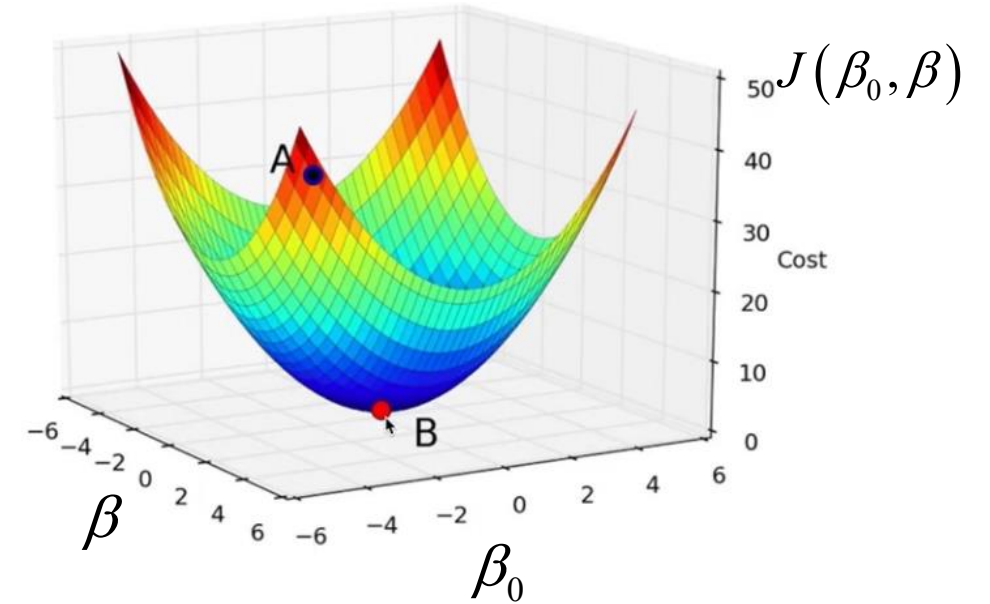
Minimizar J es entrenar el modelo.

$$y = \beta_0 + \beta x + \varepsilon \quad \Rightarrow \quad \begin{aligned} \varepsilon &= y - \beta_0 - \beta x \\ \varepsilon_i &= y_i - \beta_0 - \beta x_i \end{aligned}$$

$$\varepsilon = \sum_{i=1}^n \varepsilon_i \quad \Rightarrow \quad J = \sum_{i=1}^n (\varepsilon_i)^2$$

$$J = \sum_{i=1}^n (y_i - \beta_0 - \beta x_i)^2 \quad \Rightarrow \quad J = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$J(\beta_0, \beta)$$



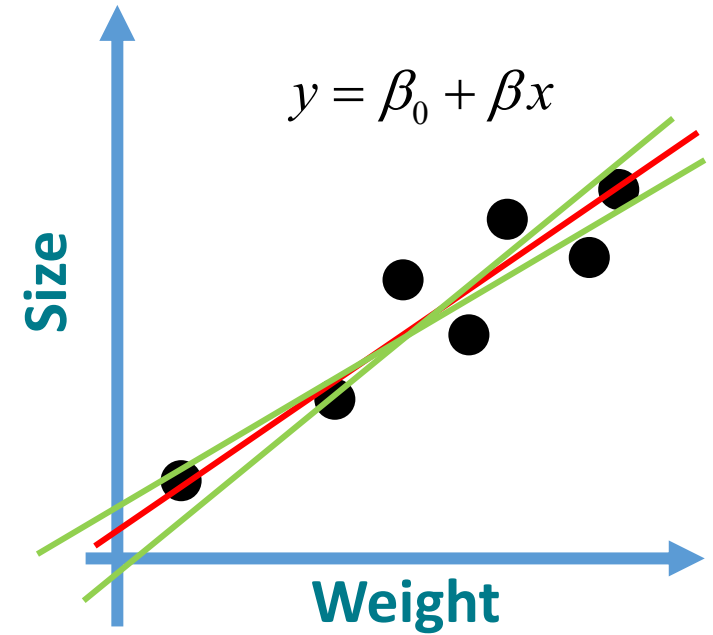
Regresión lineal

$$y = \beta_0 + \beta x + \varepsilon \Rightarrow \varepsilon = y - \beta_0 - \beta x \Rightarrow \varepsilon_i = y_i - \beta_0 - \beta x_i$$

$$\varepsilon = \sum_{i=1}^n \varepsilon_i \Rightarrow J = \sum_{i=1}^n (\varepsilon_i)^2 \Rightarrow J = \sum_{i=1}^n (y_i - \beta_0 - \beta x_i)^2 \Rightarrow J = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$\frac{\partial J}{\partial \beta_0} = -2 \sum_{i=1}^n (y_i - \beta_0 - \beta x_i) = 0 \Rightarrow \sum_{i=1}^n y_i - \sum_{i=1}^n \beta_0 - \sum_{i=1}^n \beta x_i = 0$$

$$\frac{\partial J}{\partial \beta} = -2 \sum_{i=1}^n x_i (y_i - \beta_0 - \beta x_i) = 0 \Rightarrow \sum_{i=1}^n x_i y_i - \sum_{i=1}^n \beta_0 x_i - \sum_{i=1}^n \beta x_i^2 = 0$$

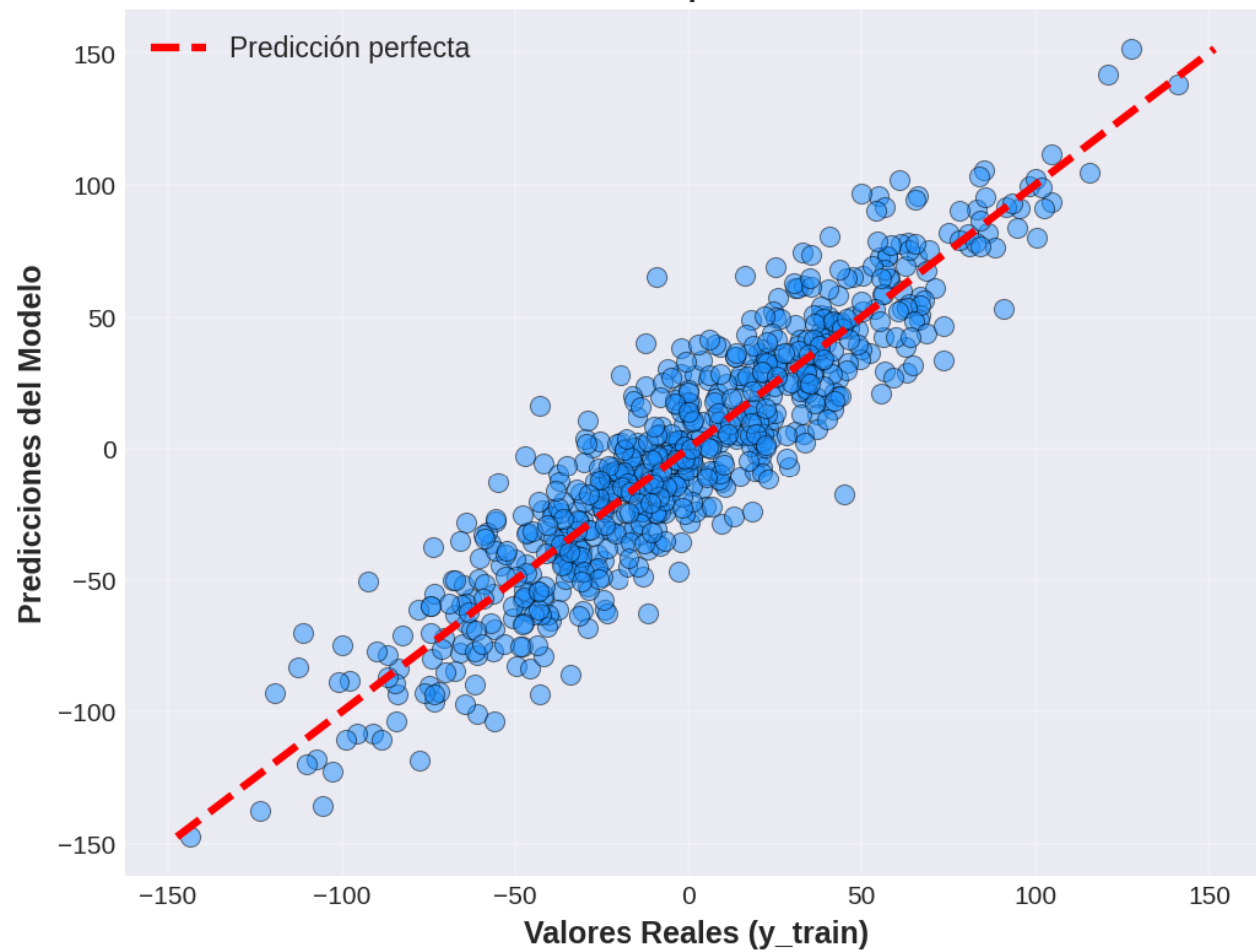


$$\beta_0 = \bar{Y} - \beta \bar{X}$$

$$\beta = \frac{n \sum_{i=1}^n x_i y_i - \sum_{i=1}^n x_i \sum_{i=1}^n y_i}{n \sum_{i=1}^n x_i^2 - \left(\sum_{i=1}^n x_i \right)^2}$$

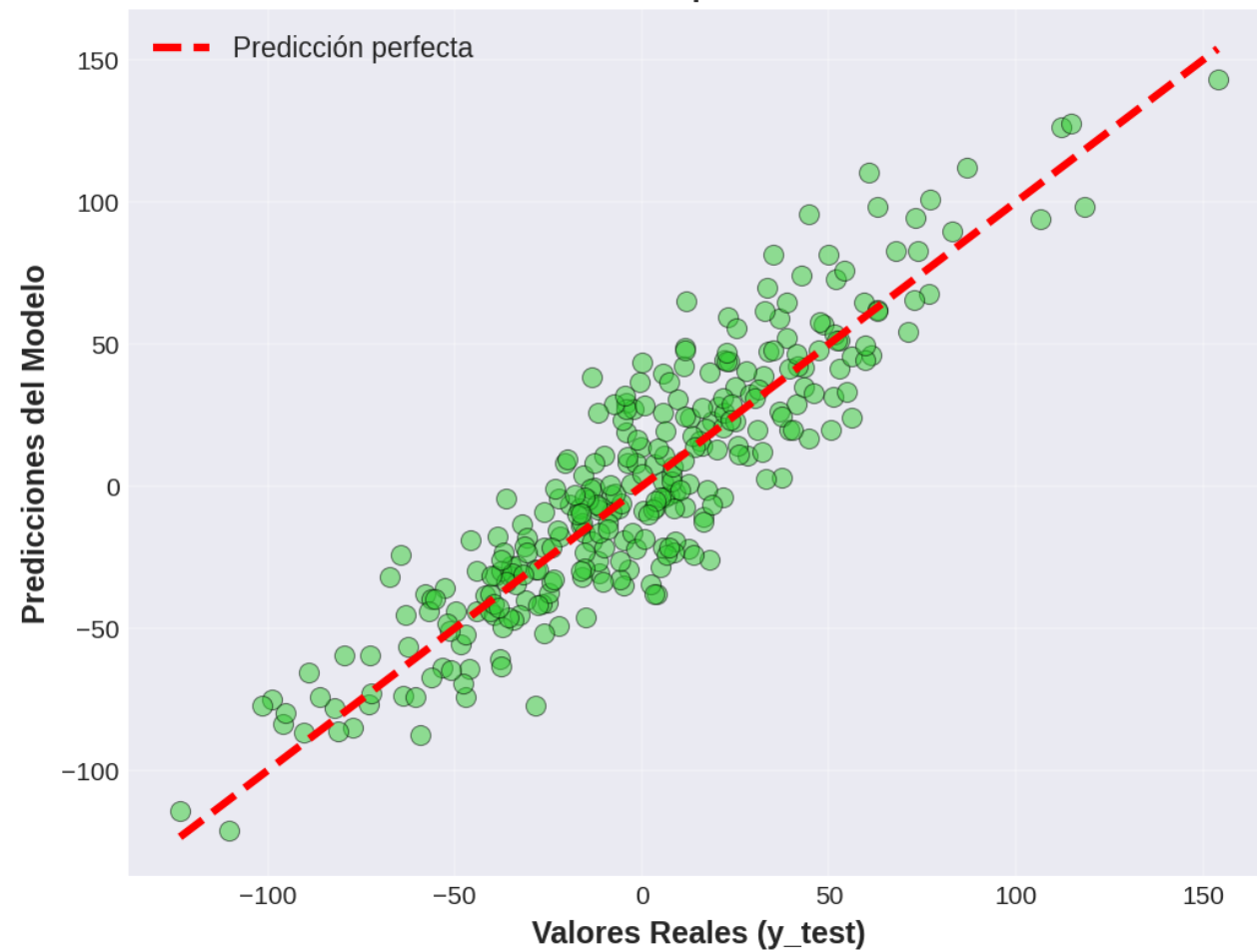
DATOS DE ENTRENAMIENTO

MSE: 359.91 | RMSE: 18.97



DATOS DE PRUEBA (NUEVOS)

MSE: 377.86 | RMSE: 19.44



Línea roja = predicción perfecta | Puntos cerca de la línea = buenas predicciones

Regresión lineal (mínimos cuadrados)

El modelo de regresión lineal que se puede usar es el siguiente:

$$\mathbf{w} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y},$$

asumiendo que el bias está incluido en $\mathbf{w}$, y la matriz de diseño tiene un vector adicional. Es posible resolver este modelo usando *ecuaciones normales*, i.e., solución analítica.

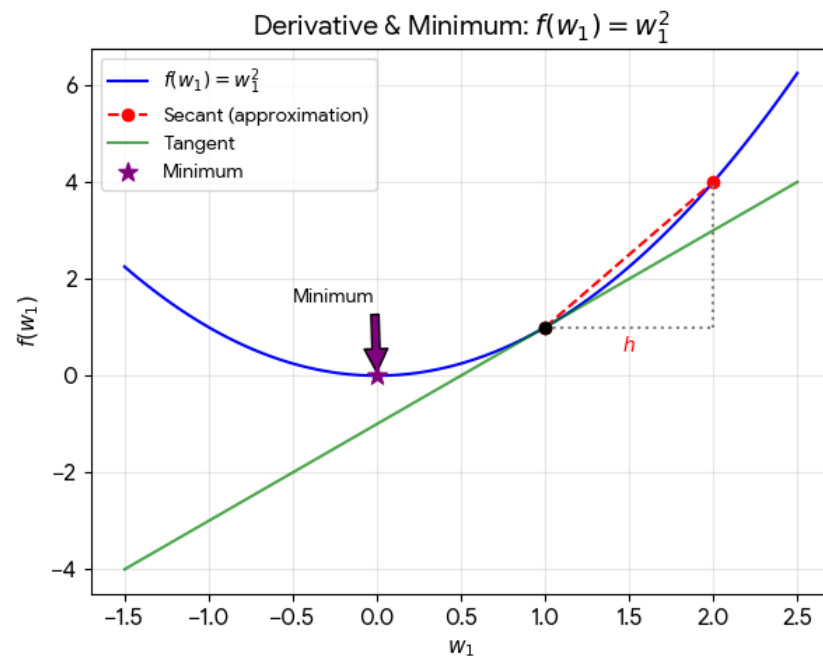
Regresión lineal (mínimos cuadrados)

Una mejor manera

Una mejor forma de hacerlo, es analizando qué efecto tiene un cambio en un parámetro en el desempeño predictivo (pérdida) del modelo; posteriormente, se cambiará solo un poco el peso en la dirección que mejora el desempeño (minimiza la pérdida). Esto se hace con bastantes pasos pequeños, hasta que la pérdida no se decremente más.

Derivadas de funciones univariadas

$$\frac{d}{dw_1} f(w_1) = \lim_{h \rightarrow 0} \frac{f(w_1 + h) - f(w_1)}{h}$$

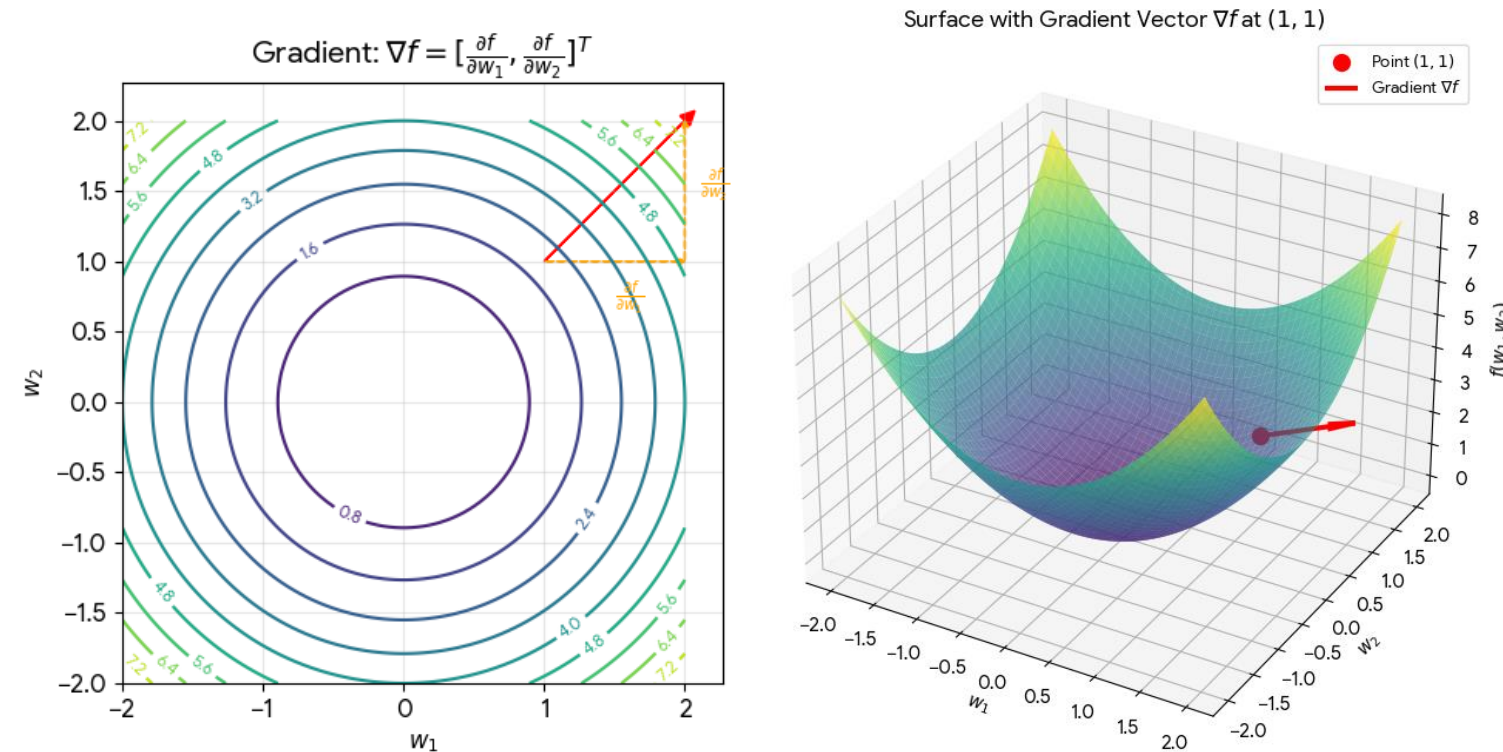


Gradiente: Derivadas de funciones multivariadas

$$\nabla f(w_1, w_2) = \begin{bmatrix} \frac{\partial f}{\partial w_1} \\ \frac{\partial f}{\partial w_2} \end{bmatrix} = \begin{bmatrix} \lim_{h \rightarrow 0} \frac{f(w_1+h, w_2) - f(w_1, w_2)}{h} \\ \lim_{h \rightarrow 0} \frac{f(w_1, w_2+h) - f(w_1, w_2)}{h} \end{bmatrix}$$

Si se tiene la función $f(x, y, z, \dots)$,

el gradiente tendrá la forma $\Delta f = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \\ \frac{\partial f}{\partial z} \\ \vdots \end{bmatrix}$.



Ejemplo

Para la función $f(x, y) = x^2y + y$ tendremos el gradiente $\Delta f = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix}$,
donde

$$\frac{\partial f}{\partial x} = \frac{\partial}{\partial x} x^2y + y = 2xy$$

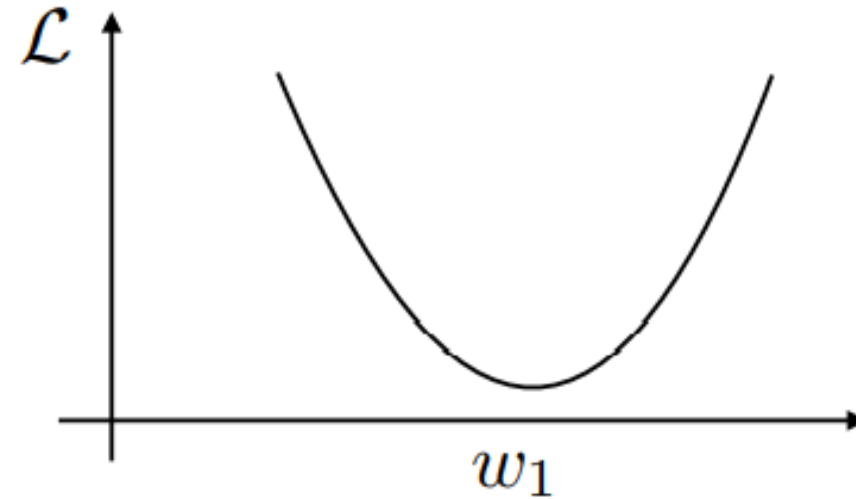
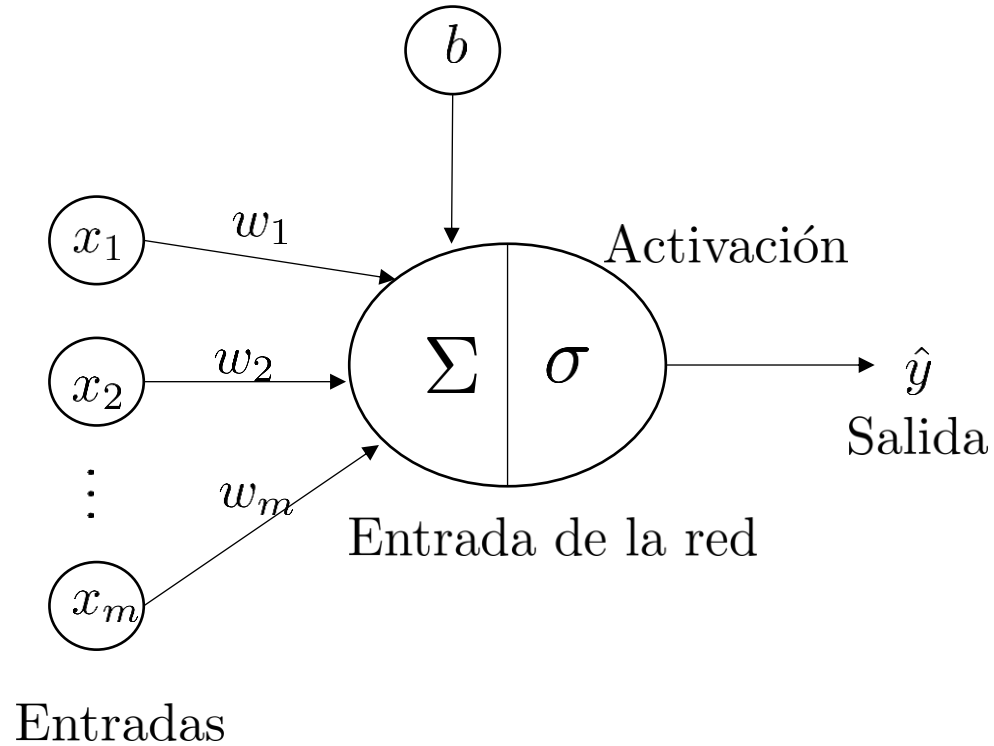
y

$$\frac{\partial f}{\partial y} = \frac{\partial}{\partial y} x^2y + y = x^2 + 1.$$

Por tanto, el gradiente de la función f estará definido como

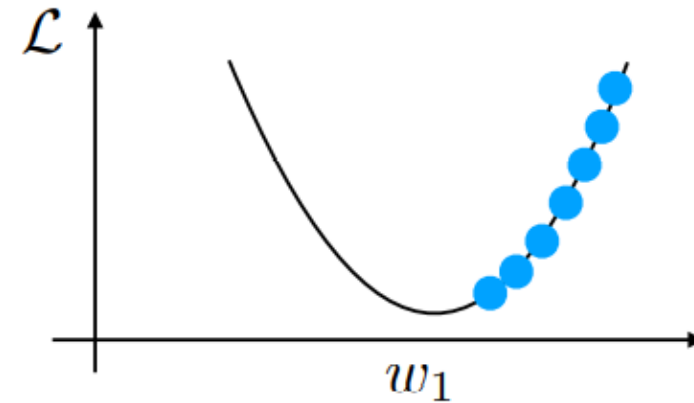
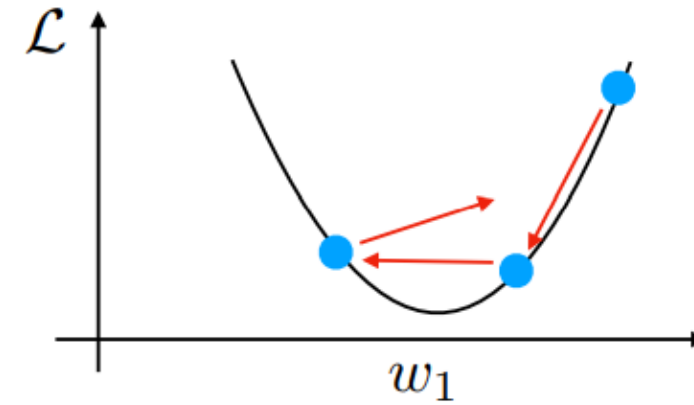
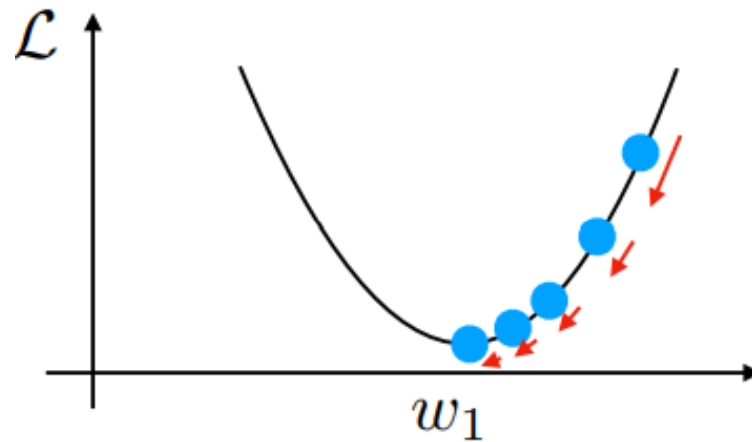
$$\Delta f(x, y) = \begin{bmatrix} 2xy \\ x^2 + 1 \end{bmatrix}$$

Entrenamiento del regresor lineal con gradiente descendente



Función de pérdida convexa
$$\mathcal{L}(\mathbf{w}, b) = \sum_i (\hat{y}^{[i]} - y^{[i]})^2$$

Entrenamiento del regresor lineal con gradiente descendente



Regla de aprendizaje del perceptrón

```
Result:  $w, b$ 
 $w := \mathbf{0} \in \mathbb{R}^m, b := 0;$ 
for  $t = 1, \dots, T$  do
  for  $i = 1, \dots, n$  do
     $\hat{y}^{[i]} := \sigma(\mathbf{x}^{[i]\top} \mathbf{w} + b);$ 
     $err := (y^{[i]} - \hat{y}^{[i]});$ 
     $\mathbf{w} := \mathbf{w} + err \times \mathbf{x}^{[i]},$ 
     $b := b + err;$ 
  end
end
```

Gradiente descendente estocástico

```
Result:  $w, b$ 
 $\mathbf{w} := \mathbf{0} \in \mathbb{R}^m, b := 0;$ 
for  $t = 1, \dots, T$  do
  for  $i = 1, \dots, n$  do
     $\hat{y}^{[i]} := \sigma(\mathbf{x}^{[i]\top} \mathbf{w} + b);$ 
     $\Delta_{\mathbf{w}} \mathcal{L} = (y^{[i]} - \hat{y}^{[i]}) \mathbf{x}^{[i]};$ 
     $\Delta_b \mathcal{L} = (y^{[i]} - \hat{y}^{[i]});$ 
     $\mathbf{w} := \mathbf{w} + \eta \times (-\Delta_{\mathbf{w}} \mathcal{L})$  ; gradiente negativo
     $b := b + \eta \times (-\Delta_b \mathcal{L});$ 
  end
end
```

tasa de aprendizaje

Vectorizado

Result: $\mathbf{w}, b$

$\mathbf{w} := \mathbf{0} \in \mathbb{R}^m, \mathbf{b} := 0;$

for $t = 1, \dots, T$ **do**

for $i = 1, \dots, n$ **do**

$\hat{y}^{[i]} := \sigma(\mathbf{x}^{[i]\top} \mathbf{w} + b);$

$\Delta_{\mathbf{w}} \mathcal{L} = (y^{[i]} - \hat{y}^{[i]}) \mathbf{x}^{[i]} ;$

$\Delta_b \mathcal{L} = (y^{[i]} - \hat{y}^{[i]});$

$\mathbf{w} := \mathbf{w} + \eta \times (-\Delta_{\mathbf{w}} \mathcal{L}) ;$

$b := b + \eta \times (-\Delta_b \mathcal{L});$

end

end

Ciclo *for*

Result: $\mathbf{w}, b$

$\mathbf{w} := \mathbf{0} \in \mathbb{R}^m, \mathbf{b} := 0;$

for $t = 1, \dots, T$ **do**

$\Delta \mathbf{w} := \mathbf{0}, \Delta b := 0;$

for $i = 1, \dots, n$ **do**

$\hat{y}^{[i]} := \sigma(\mathbf{x}^{[i]\top} \mathbf{w} + b);$

end

for $j = 1, \dots, m$ **do**

$\frac{\partial \mathcal{L}}{\partial w_j} = (y^{[i]} - \hat{y}^{[i]}) x_j^{[i]};$

$w_j := w_j + \eta \times \left(-\frac{\partial \mathcal{L}}{\partial w_j}\right);$

end

$\frac{\partial \mathcal{L}}{\partial b} = (y^{[i]} - \hat{y}^{[i]});$

$b := b + \eta \times \left(-\frac{\partial \mathcal{L}}{\partial b}\right);$

end

Modo *On-line*

```
Result:  $\mathbf{w}, b$ 
 $\mathbf{w} := \mathbf{0} \in \mathbb{R}^m, \mathbf{b} := 0;$ 
for  $t = 1, \dots, T$  do
   $\Delta \mathbf{w} := 0, \Delta b := 0;$ 
  for  $i = 1, \dots, n$  do
     $\hat{y}^{[i]} := \sigma(\mathbf{x}^{[i]\top} \mathbf{w} + b);$ 
  end
  for  $j = 1, \dots, m$  do
     $\frac{\partial \mathcal{L}}{\partial w_j} = (y^{[i]} - \hat{y}^{[i]})x_j^{[i]};$ 
     $w_j := w_j + \eta \times \left(-\frac{\partial \mathcal{L}}{\partial w_j}\right);$ 
  end
   $\frac{\partial \mathcal{L}}{\partial b} = (y^{[i]} - \hat{y}^{[i]});$ 
   $b := b + \eta \times \left(-\frac{\partial \mathcal{L}}{\partial b}\right);$ 
end
```

Coincidentalmente, parece casi el mismo de la regla del perceptrón, exceptuando que la predicción es un número real y se tiene una tasa de aprendizaje. Esta regla de aprendizaje se llama **gradiente descendente estocástico**

Dada la función de pérdida

$$\mathcal{L}(\mathbf{w}, b) = \sum_i (\hat{y}^{[i]} - y^{[i]})^2,$$

la derivada estaría dada por

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial w_j} &= \frac{\partial}{\partial w_j} \sum_i (\hat{y}^{[i]} - y^{[i]})^2 \\ &= \frac{\partial}{\partial w_j} \sum_i (\sigma(\mathbf{w}^\top \mathbf{x}^{[i]}) - y^{[i]})^2 \\ &= \sum_i 2(\sigma(\mathbf{w}^\top \mathbf{x}^{[i]}) - y^{[i]}) \frac{\partial}{\partial w_j} (\sigma(\mathbf{w}^\top \mathbf{x}^{[i]}) - y^{[i]}) \\ &= \sum_i 2(\sigma(\mathbf{w}^\top \mathbf{x}^{[i]}) - y^{[i]}) \frac{d\sigma}{d(\mathbf{w}^\top \mathbf{x}^{[i]})} \frac{\partial}{\partial w_j} \mathbf{w}^\top \mathbf{x}^{[i]} \\ &= \sum_i 2(\sigma(\mathbf{w}^\top \mathbf{x}^{[i]}) - y^{[i]}) \frac{d\sigma}{d(\mathbf{w}^\top \mathbf{x}^{[i]})} x_j^{[i]} \\ &= \sum_i 2(\sigma(\mathbf{w}^\top \mathbf{x}^{[i]}) - y^{[i]}) x_j^{[i]} \end{aligned}$$

Note que la función de activación para el caso de la regresión lineal es la función identidad.

Derivada de la función de pérdida para regresión lineal: caso alternativo

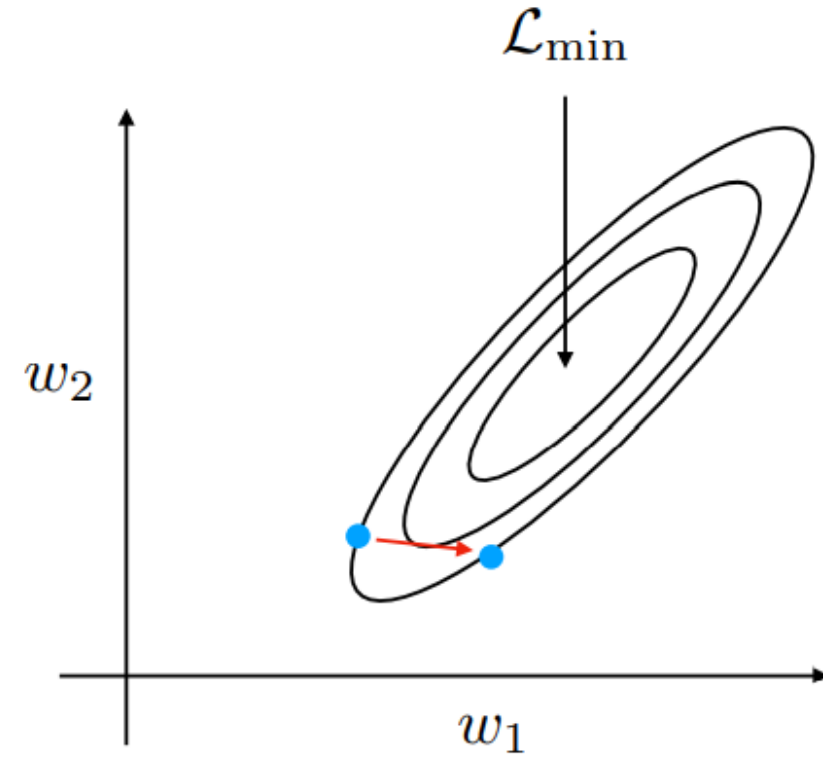
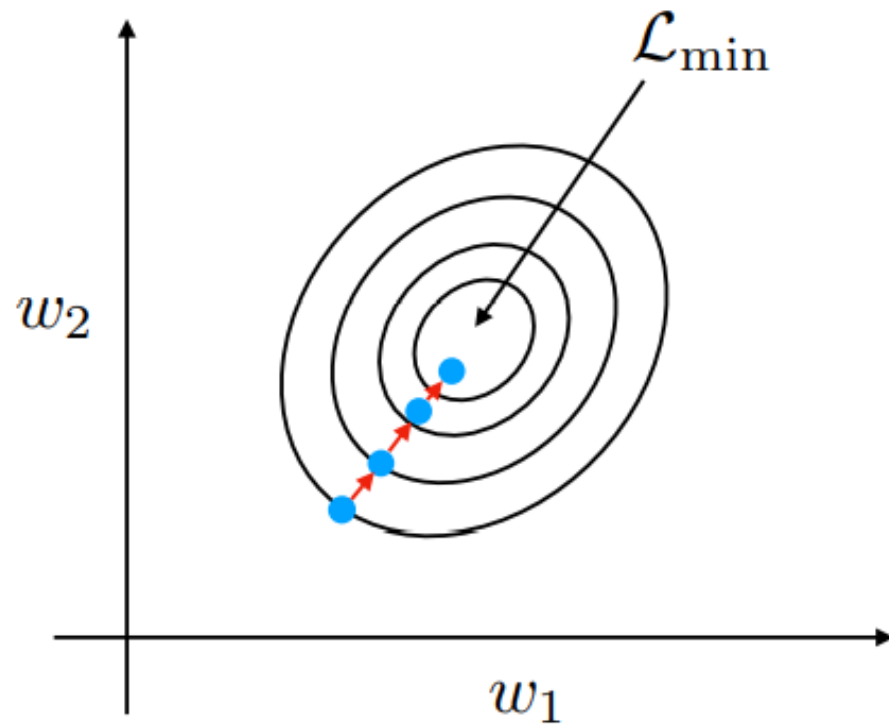
A menudo, la función de pérdida se escala por un factor de $1/2$ por conveniencia:

$$\mathcal{L}(\mathbf{w}, b) = \frac{1}{2n} \sum_i (\hat{y}^{[i]} - y^{[i]})^2.$$

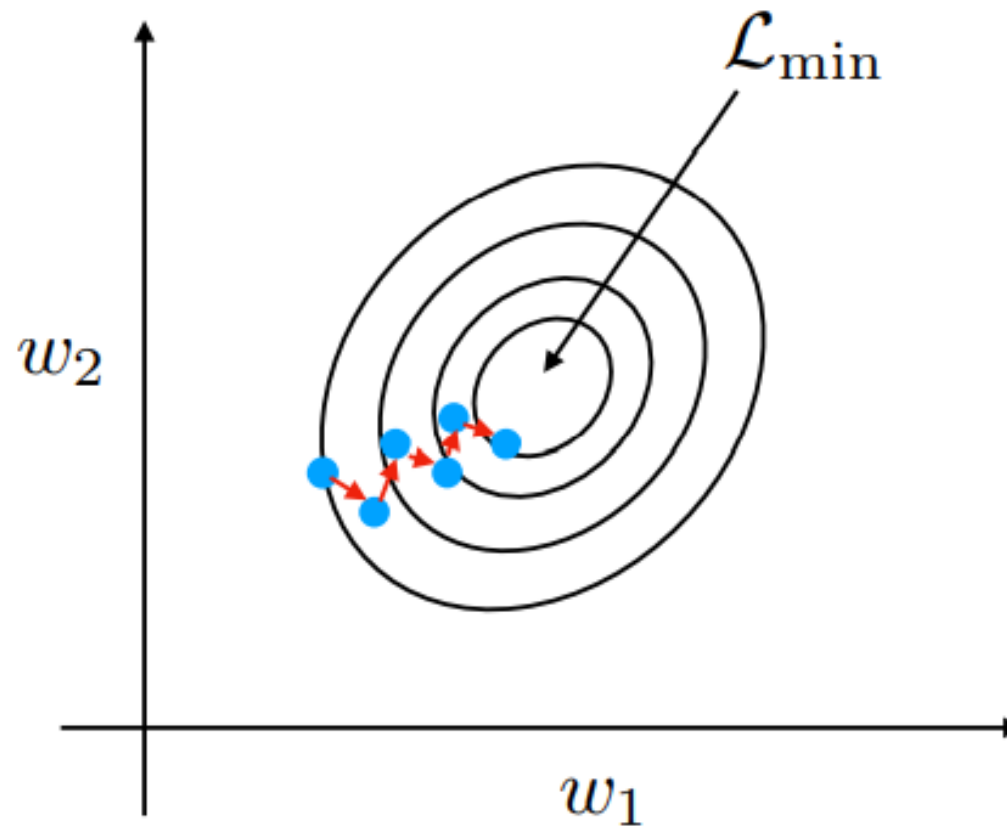
La derivada estaría dada por

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial w_j} &= \frac{\partial}{\partial w_j} \frac{1}{2n} \sum_i (\hat{y}^{[i]} - y^{[i]})^2 \\ &= \frac{\partial}{\partial w_j} \sum_i \frac{1}{2n} (\sigma(\mathbf{w}^\top \mathbf{x}^{[i]}) - y^{[i]})^2 \\ &= \sum_i \frac{1}{n} (\sigma(\mathbf{w}^\top \mathbf{x}^{[i]}) - y^{[i]}) \frac{\partial}{\partial w_j} (\sigma(\mathbf{w}^\top \mathbf{x}^{[i]}) - y^{[i]}) \\ &= \sum_i \frac{1}{n} (\sigma(\mathbf{w}^\top \mathbf{x}^{[i]}) - y^{[i]}) \frac{d\sigma}{d(\mathbf{w}^\top \mathbf{x}^{[i]})} \frac{\partial}{\partial w_j} \mathbf{w}^\top \mathbf{x}^{[i]} \\ &= \sum_i \frac{1}{n} (\sigma(\mathbf{w}^\top \mathbf{x}^{[i]}) - y^{[i]}) \frac{d\sigma}{d(\mathbf{w}^\top \mathbf{x}^{[i]})} x_j^{[i]} \\ &= \sum_i \frac{1}{n} (\sigma(\mathbf{w}^\top \mathbf{x}^{[i]}) - y^{[i]}) x_j^{[i]} \end{aligned}$$

Gradiente descendente por lotes



Gradiente descendente estocástico



Entrenamiento de una red neuronal de una sola capa con gradiente descendente

Widrow and Hoff's ADALINE (1960)

A nicely differentiable neuron model

Widrow, B., & Hoff, M. E. (1960). *Adaptive switching circuits* (No. TR-1553-1). Stanford Univ Ca Stanford Electronics Labs.

Widrow, B. (1960). *Adaptive" adaline" Neuron Using Chemical" memistors."*.

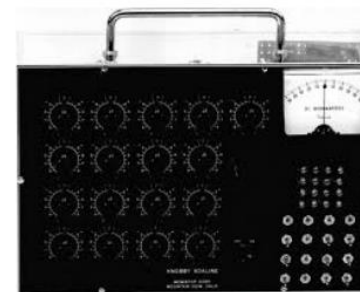
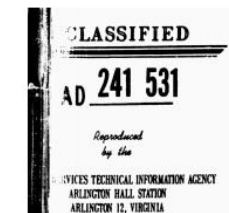


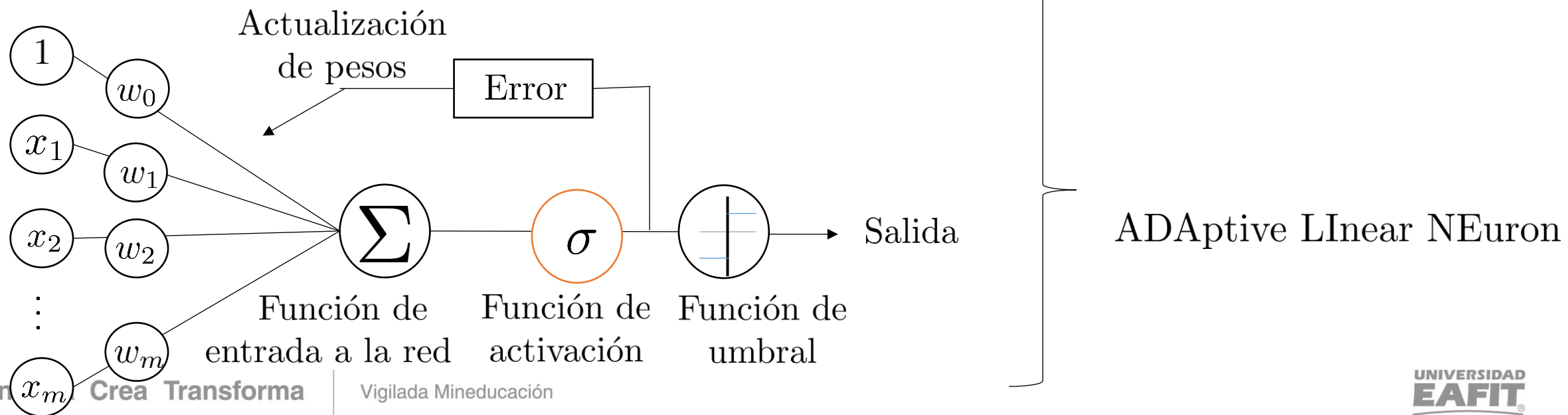
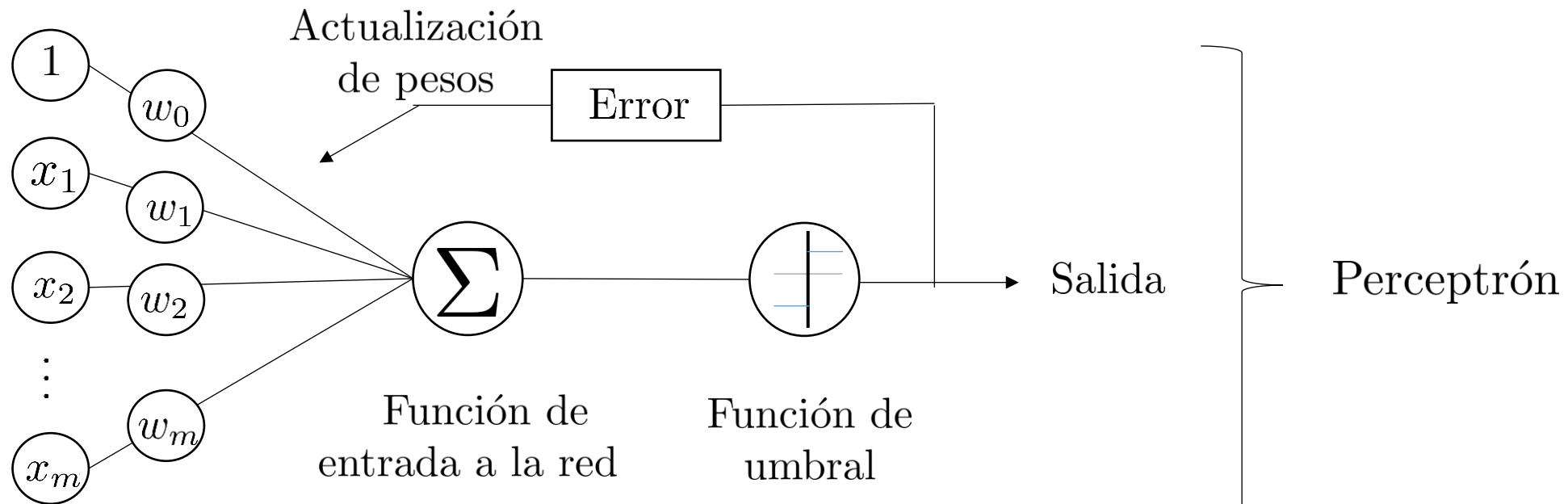
Image source: https://www.researchgate.net/profile/Alexander_Magoun2/publication/265789430/figure/fig2/AS:630233355178778001470551421849/ADALINE-An-adaptive-linear-neuron-Manually-adapted-synapses-Designed-and-built-by-Ted.png



THIS REPORT HAS BEEN DELIMITED
AND CLEARED FOR PUBLIC RELEASE
UNDER DOD DIRECTIVE 5200.20 AND
NO RESTRICTIONS ARE IMPOSED UPON
ITS USE AND DISCLOSURE.

DISTRIBUTION STATEMENT A

APPROVED FOR PUBLIC RELEASE;
DISTRIBUTION UNLIMITED.



Algunos tutoriales sobre PyTorch

- <https://pytorch.org/tutorials/>
- https://pytorch.org/tutorials/beginner/deep_learning_60min_blitz.html
- <https://pytorch.org/assets/deep-learning/Deep-Learning-with-PyTorch.pdf>

y el foro de discusión

<https://discuss.pytorch.org/>

